

Article

Large Language Model Based Intelligent Fault Information Retrieval System for New Energy Vehicles

Haiyu Zhang ¹, Yinghui Zhao ², Boyu Sun ², Yaqi Wu ¹, Zetian Fu ^{2,*} and Xinqing Xiao ^{2,*}¹ Yantai Institute, China Agricultural University, Yantai 264670, China; zhanghaiyu@cau.edu.cn (H.Z.); 19558583807@163.com (Y.W.)² College of Engineering, China Agricultural University, Beijing 100083, China; 18334577924@163.com (Y.Z.)

* Correspondence: fzt@cau.edu.cn (Z.F.); xxqjd@cau.edu.cn (X.X.)

Abstract: In recent years, the rapid development of the new energy vehicle (NEV) industry has exposed significant deficiencies in intelligent fault diagnosis and information retrieval technologies, especially in intelligent fault information retrieval, which faces persistent challenges including inadequate system adaptability and reasoning bottlenecks. To address these challenges, this study proposes a Retrieval-Augmented Generation (RAG) framework that integrates large language models (LLMs) with knowledge graphs (KGs). The framework consists of three key components: fault data collection, knowledge graph construction, and fault knowledge model training. The primary research contributions are threefold: (1) A domain-optimized fine-tuning strategy for LLMs based on NEV fault characteristics, verifying the superior accuracy of the Bidirectional Encoder Representations from Transformers (BERT) model in fault classification tasks. (2) A structured knowledge graph encompassing 122 fault categories, developed through the ChatGLM3-6B model completing named entity and knowledge relation extraction to generate fault knowledge and build a paraphrased vocabulary. (3) An intelligent fault information retrieval system that significantly outperforms traditional models in NEV-specific Q&A scenarios, providing multi-level fault cause analysis and actionable solution recommendations.

Academic Editor: Jose María Alvarez Rodríguez

Received: 27 February 2025

Revised: 1 April 2025

Accepted: 3 April 2025

Published: 6 April 2025

Citation: Zhang, H.; Zhao, Y.; Sun, B.; Wu, Y.; Fu, Z.; Xiao, X. Large Language Model Based Intelligent Fault Information Retrieval System for New Energy Vehicles. *Appl. Sci.* **2025**, *15*, 4034. <https://doi.org/10.3390/app15074034>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

Keywords: new energy vehicle fault; intelligent retrieval system; generative language models; knowledge graphs; retrieval-augmented generation

1. Introduction

Currently, China is becoming a global leader in the research, development, production, sales, and maintenance of NEVs, maintaining the world's largest number of NEVs for several consecutive years. Along with the continuous evolution of digitalization and intelligent technologies in NEVs, a large number of new platforms and new vehicle fault evaluation indicators are also emerging. This rapid evolution is bringing about a series of new challenges, such as the complexity of functional layouts and configurations in NEVs, the reliability and stability of charging infrastructure, and consumers' insufficient understanding of potential fault causes, manifestations, and consequences in NEVs [1].

Due to significant differences in the technology of NEVs compared to traditional gasoline vehicles and the complexity of the three-electric system, consumers often struggle to cope with the integrated and complex nature of NEVs. These issues lead to difficulties for ordinary consumers in addressing complex usage situations, fault maintenance, and

relevant policy regulations. Currently, fault diagnosis technologies in this field still face the following challenges: (1) Insufficient data and fragmented information: Current fault diagnosis methods rely on large amounts of data to establish signal features or model parameters. However, the diversity of NEV brands and models makes data standardization challenging. Furthermore, the incomplete knowledge system and unstructured data impose limitations on expert systems based on system knowledge or rules for fault reasoning [2]. (2) Low information integration: Although significant breakthroughs have been made in knowledge reasoning and intelligent generation through large language models (LLMs) and knowledge graphs (KGs) in recent years, the field of NEVs still lacks systematic and comprehensive fault knowledge. The issue of fragmented information remains severe [3]. (3) Challenges in building specialized LLMs and knowledge bases for NEVs. Developing specialized LLMs and knowledge bases for NEVs faces many difficulties. In the context of information fragmentation and incomplete knowledge system structures, intelligently training models and generating knowledge require breakthroughs in methods and technologies.

To address these issues, this study proposes a method that combines fault tree analysis with an expert system for fault diagnosis. The knowledge base constructed through fault tree analysis not only contains the causal relationships of NEV faults and solutions but also retains the unique hierarchical relationships between fault tree knowledge conclusions. By converting the fault tree into a knowledge structure combining KGs and rules, this method can effectively support reasoning and analysis in the fault diagnosis process. This method not only addresses current technological bottlenecks but also provides a sustainable solution for NEV fault diagnosis.

The main contributions of this study include:

- Proposing an innovative electric vehicle fault information retrieval system that integrates large-scale language models and KGs, capable of efficiently managing diverse and fragmented fault data;
- Developing a knowledge graph covering 122 different fault categories and using the ChatGLM3-6B model to promote efficient extraction and reasoning of fault knowledge;
- Creating a fault diagnosis system based on RAG technology that provides accurate fault cause analysis and offers feasible solution recommendations.

This paper is divided into six chapters. Section 1 provides an introduction, which briefly outlines the background and research significance of intelligent fault information retrieval systems. Section 2 analyzes the current research trends and functional requirements of intelligent fault information retrieval systems, focusing on global research trends in fault diagnosis and information retrieval systems, as well as the research directions of knowledge graphs and generative language models for NEVs. Section 3 designs the architecture of the intelligent fault information retrieval system, including the logical structure design of the fault data retrieval system and its framework. Section 4 presents the data acquisition and preprocessing process, model training for new energy vehicle faults, as well as system demonstration and testing. Section 5 discusses the fine-tuning of knowledge graphs and large language models and explores key issues in fault data retrieval and system interaction. Section 6 summarizes the main contributions of the paper and outlines future research directions.

2. Current Trends and Functional Requirements Analysis of Intelligent Fault Information Retrieval Systems for NEVs

2.1. Global Research Trends

2.1.1. Fault Diagnosis and Information Retrieval Systems Research Trends

In recent years, the development of fault diagnosis technologies for automobiles and integrated electromechanical equipment has progressed rapidly and can generally be divided into three types: signal-based fault diagnosis [4,5], knowledge-based fault diagnosis [6,7], and model-based fault diagnosis [8]. The classification diagram of fault diagnosis technology is shown in Figure 1. With advancements in artificial intelligence technology, increasing attention is being paid to combining model-driven and data-driven approaches. For example, Qiao et al. [9] applied stochastic resonance (SR) to mechanical fault detection. Unlike traditional signal processing methods, stochastic resonance utilizes noise embedded within signals to extract weak fault features, making it widely used in mechanical fault detection. Xiao et al. [10] introduced a new expert system knowledge base formed by combining the dynamic fault tree model with the expert system knowledge base. Zheng et al. [11] introduced a knowledge transfer strategy for cross-domain fault diagnosis, highlighting the application of transfer learning in constructing efficient fault diagnosis models. Jafari et al. [12] proposed an online lithium-ion battery state estimation method based on the extreme gradient boosting (XGBoost) algorithm. This method uses voltage-time data from partial constant current stages as input and obtains a nonlinear relationship model through offline training, thereby improving the accuracy of battery state estimation in electric vehicle applications. Xia et al. [13] proposed a real-time fault detection and process control method based on multi-channel sensor data fusion. Using multi-channel sensor data fusion and uncorrelated multilinear discriminant analysis (UMLDA) to model multi-channel data, the method achieved superior monitoring and fault diagnosis performance compared to other approaches. Simulation and case analysis demonstrated the effectiveness of this method in rapidly detecting equipment faults. In reality, automotive automatic diagnosis and expert system fault diagnosis methods maintain high usage rates and have become fundamental means of fault diagnosis. With the development of network technologies, digital communication, and artificial intelligence, integrated fault diagnosis systems that combine networks and artificial intelligence are becoming the new trend leading the field of automotive fault diagnosis [14–16].

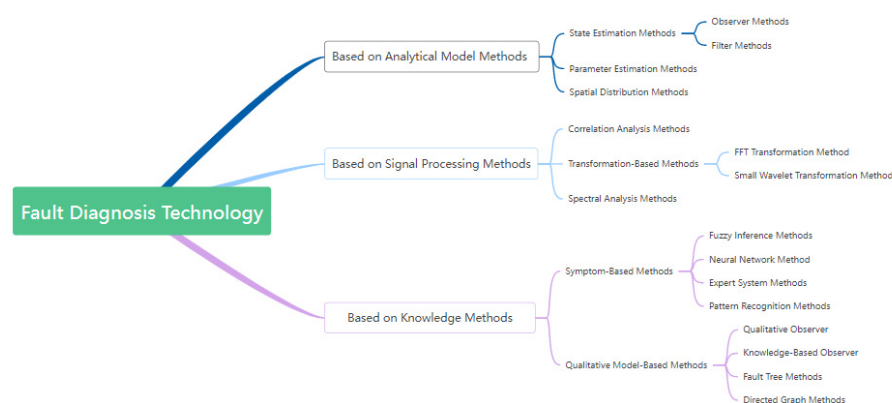


Figure 1. The classification diagram of fault diagnosis technology.

2.1.2. Knowledge Graphs and Generative Language Models Research Trends

(1) Generative Language Models

Since the introduction of the Transformer model in 2017, various models based on the Transformer have emerged: The BERT model is primarily used for traditional NLP tasks [17,18], while the GPT (Generative Pre-trained Transformer) model is mainly used for text generation tasks. Currently, various image processing methods based on Transformer architecture have emerged, making significant progress [19].

GPT is a generative pre-trained model based on the Transformer architecture and was proposed by OpenAI in 2018. It serves as the foundation for all LLMs and is a model capable of generating natural language text based on a given context. Transformer is a deep neural network architecture based on the attention mechanism, which can effectively capture long-distance dependencies in text and has high parallelism and scalability. The GPT models have made groundbreaking advancements in natural language processing and have demonstrated powerful capabilities in text generation tasks, such as generating coherent and diverse follow-up text [20–22].

Generative language models like GPT have shown broad application prospects in various fields, generating fluent, coherent, and logical text, and handling multimodal input of images and text. For example, GPT-4, as a large-scale multimodal model, can accept both image and text as input and generate text as output. Its performance surpasses previous language models and can complete difficult tasks across multiple fields. Furthermore, GPT-4 significantly outperforms existing language models in traditional natural language processing tasks, even though these systems are usually fine-tuned for specific task benchmarks [23,24].

(2) Knowledge Graphs and Applications

A knowledge graph is a technology that represents and organizes structured knowledge through a graphical model, containing rich semantic information between entities, relationships, and attributes. Its application is of great significance to expert systems, providing rich background knowledge and semantic associations to help answer user questions. The construction methods of KGs have always attracted attention. Traditional methods mainly rely on manual annotation or knowledge extraction from structured data. In recent years, methods based on natural language processing and machine learning have gradually emerged, enabling automatic extraction of knowledge from unstructured text data, and improving the efficiency and coverage of knowledge graph construction.

Currently, the applications of KGs can be categorized into four main types: the first is methods based on question-answer templates, the second is research conducted through semantic parsing, the third is methods using deep learning to rank answers, and the last is research conducted using knowledge graph embedding learning. In recent years, the continuous progress of KGs and their deep application in various fields have made knowledge graph-based information systems a growing research focus [25].

(3) The Combination of Generative Language Models and KGs

Large models have demonstrated excellent performance in various natural language processing tasks, but they still have significant limitations when facing complex knowledge reasoning tasks, such as data biases, fairness issues, poor interpretability, transparency, and factual inaccuracies leading to model hallucinations. To address these problems, researchers are improving training algorithms, proposing new model architectures, using more fair and balanced datasets, increasing model transparency and interpretability, and developing more effective knowledge integration techniques [26,27].

RAG emerged as a novel knowledge fusion solution that seamlessly weaves information retrieval into the prompt process. RAG analyzes user input to create a target query

and search for relevant resources in a pre-built knowledge base. The retrieved fragments are merged into the original prompt and enriched with the contextual background. Typical RAG techniques, such as LangChain split the text into chunks and use encoding models to embed them into vectors and store the vectors in an index. When a user initiates a query, the same encoding model vectorizes the query. Then, a search is performed on the index to retrieve relevant text chunks, which are used as contextual input to the generative model.

A knowledge base can be a structured database, unstructured textual literature, or a knowledge graph database. In the application of combining generative language models with KGs, besides embedding graph data into model training and enhancing models with graphs, the approaches shown in Table 1 are widely used.

Table 1. The integration methods of KGs and LLMs.

Method	Core Focus	Limitation	Reference Examples
Pure LLM Reasoning	Using the idea of linking, LLMs can solve some reasoning problems	Difficult to handle professional knowledge-intensive reasoning and complex reasoning tasks [28]	ChatGPT4.0, ChatGPT3.5, ChatGLM, Wenxin-Yiyan, etc.
$LLM \oplus KG$	Large models play various roles, through querying knowledge in KG to enhance reasoning capabilities, enabling the addition of external knowledge into the model	Embedding LLM into KG limits its interpretability and introduces complexity when updating the knowledge base	Li et al. used LLM to generate SPARQL queries for KGs, and the main subject of the study is to complete KGs [29]
$LLM \otimes KG$	Cooperation between KG and LLM, enabling knowledge graph completion and reasoning through both LLM and KG integration	Needing to consider the accurate path to knowledge graph completion while integrating external knowledge	Sun et al. used KG/LLM in the beam search algorithm for knowledge reasoning, enabling enhanced path expansion for KG generation [30]

To address the limitations outlined in Table 1, this study integrates large language LLMs and KGs to enhance fault diagnosis for NEVs. Unlike traditional methods, which struggle with complex reasoning and real-time knowledge updates, our approach leverages retrieval-augmented generation RAG to enable more context-aware fault identification. The combination of ChatGLM3-6B and a structured fault knowledge graph allows for efficient knowledge extraction and multi-hop reasoning, improving both accuracy and adaptability.

2.2. Functional Requirements of Intelligent Fault Information Retrieval Systems for NEVs

NEVs are essentially complex electromechanical systems in motion. Their fault information retrieval systems must fulfill multiple functions, including fault information acquisition, knowledge-based Q&A on faults, and maintenance guidance.

To fully understand the characteristics and patterns of fault information retrieval for NEVs, this study designed a questionnaire survey targeting different users and market sales personnel, focusing on clarifying the following key issues.

- (1) Basic user needs: This includes understanding the aspects of NEV-related fault knowledge that different users prioritize; whether they have reliable methods and resources for obtaining information on NEV usage, maintenance, and upkeep; their preferred channels for accessing information when faults occur and the reliability of these channels, such as whether they can help users promptly identify and locate issues; users' needs for rapid fault diagnosis tools and their trust in maintenance suggestions. Additionally, it investigates the performance of existing NEV mobile

applications and automotive apps in meeting users' daily fault information retrieval needs and whether these platforms and applications can provide users with fast and convenient services.

- (2) User preferences for fault information retrieval: For example, whether users prefer using search engines to find possible fault causes and solutions; their awareness of and specific requirements for professional intelligent search systems; whether traditional automotive fault diagnosis systems are difficult to use or require high technical expertise; the common challenges users face in accessing NEVs fault data; and their systematic needs for fault information retrieval.
- (3) Requirements for system functionality: Whether existing fault diagnosis expert systems can provide rapid analysis of fault information for different NEVs; whether they meet the needs of users who lack professional knowledge; whether the knowledge base of NEVs faults within these systems is comprehensive; whether existing automotive mobile applications support fault consultation and provide problem-solving suggestions; whether these applications present NEVs faults and solutions in an intuitive and user-friendly way; and whether the system platforms suffer from issues such as overly complicated interfaces, inefficiency in responding to user inquiries, or low responsiveness that negatively impact the user experience.

Through user surveys, literature analysis, and an evaluation of existing automotive client applications, it has been found that current fault information retrieval systems for NEVs face significant challenges in meeting the diverse needs of different users. The primary issues include the following:

- (1) The user interfaces of existing automotive fault diagnosis expert systems are overly complex and crude, and there is limited information specifically related to NEVs.
- (2) Most systems are unable to conduct precise reasoning and analysis of automotive faults. The fault diagnosis technology they rely on requires a large amount of fault data to establish signal features or model parameters. For the vast variety of NEVs currently available, this is difficult to achieve under current conditions. Moreover, it is challenging to unify fault data for different models of NEVs. While knowledge-based reasoning methods do not require extensive fault data, they also face certain difficulties. For example, they can only perform fault detection within the scope of existing knowledge, and acquiring new knowledge is challenging, with incomplete representation of knowledge being another issue.
- (3) Existing automotive fault diagnosis systems are cumbersome to operate. Each vehicle model requires individual analysis, and the interfaces of both client applications and web pages are disorganized and cluttered.

Table 2 summarizes the problems and requirements of the system. By addressing the existing issues at various levels of these requirements, the direction for the construction of the NEVs fault data retrieval system has been determined.

Table 2. Survey of existing system problems.

Serial Number	Existing Problems in the System	Brief Description	System Requirements	New System Requirements
1	Information search is inconvenient	The client-side web page refreshes, the interface is not user-friendly, and there is no focus on NEV fault optimization	Customer-side retrieval is complicated, and the interface is inconvenient	The interface should be concise, user-friendly, and able to quickly locate the required functions

2	No specific model for vehicle faults	There is no specific fault diagnosis system for vehicles	Technical method	Needs a fault detection model for vehicle fault diagnosis, providing precise and natural interaction for troubleshooting
3	Lack of relevant knowledge	NEV fault knowledge is scattered and unorganized	Data acquisition	Needs to organize knowledge on NEVs faults and establish a knowledge base with feedback.
4	Data resource scarcity	Needs to build a knowledge base for NEV faults	Knowledge base	Needs to build a specialized knowledge base for NEVs with capabilities for easy integration and expert use

To address the above issues, an intelligent retrieval system for NEV fault information must achieve breakthroughs in the following areas:

- (1) Data acquisition: Intelligent methods should be used to construct a dataset specifically focused on the domain of NEV faults; additionally, a large amount of unstructured and semi-structured data scattered across different sources and literature should be systematically integrated.
- (2) Knowledge database layer: Current fault diagnosis expert systems based on fault tree analysis methods lack quick and convenient maintenance and knowledge acquisition capabilities; automotive-related websites and applications, such as Autohome and DCar, provide users with basic data like vehicle configuration parameters but fail to deliver sufficient data to comprehensively integrate knowledge in the field of NEVs. Therefore, it is necessary to construct databases and presentation methods tailored to the characteristics of NEV knowledge data. This process should involve maintaining data by model, based on common fault types and phenomena, to ultimately form a comprehensive knowledge set for NEVs faults.
- (3) Technical methods: Current automotive fault diagnosis technologies are unable to perform precise reasoning and analysis for NEV faults, as they rely heavily on large amounts of fault data to establish signal features or model parameters, which is difficult to achieve given the relatively short time NEVs have existed and their diverse range of models. Although knowledge-based reasoning methods do not require extensive fault data, they have limitations, such as their inability to detect faults beyond the existing knowledge base, difficulties in acquiring new knowledge, and incomplete knowledge representation. To address these challenges, it is necessary to employ artificial intelligence, expert system methods, and knowledge graph technologies, such as RAG for LLMs, Prompt engineering [31,32], fine-tuning of LLMs [33], BERT classification algorithms, Cypher query language, front-end visualization, and API streaming for LLMs. Through the systematic integration of these technologies, an intelligent fault diagnosis and analysis platform for NEVs can be established.
- (4) System presentation: Traditional fault diagnosis expert systems have relatively simple user interface designs, while existing automotive website platforms suffer from overly complex and cluttered page information, with their main content dominated by conventional automobiles and related data. Additionally, pages designed for user feedback on fault issues are poorly structured, making information retrieval difficult. To resolve these issues, new technologies should be implemented, such as the Flask framework combined with visualization techniques, to develop a fault data retrieval system specifically for NEVs. This system should enable intuitive visualization of the NEV faults knowledge graph, providing users with a more user-friendly and convenient service for fault diagnosis and information retrieval.

3. System Design of Intelligent Retrieval of Fault Information

3.1. The Logical Structure Design of the Fault Data Retrieval System

3.1.1. The Integration of Fault Diagnosis Technology

The Fault Tree Analysis (FTA) based on the Fault Tree Model and the Fault Diagnosis Expert System are two important branches in the field of fault diagnosis. Exploring their integration to meet system requirements.

(1) Fault Tree Analysis

Fault Tree Analysis (FTA), also known as Causal Tree Analysis, combines the advantages of qualitative analysis, rule-based reasoning, and quantitative analysis, making it a simple yet effective analysis method. It models the fault object as a fault tree, identifying and determining all possible patterns that could lead to the top-level fault, and is a powerful tool for reliability analysis.

The qualitative analysis in FTA focuses on evaluating the importance of events, i.e., analyzing the extent to which each event influences the occurrence of the top-level fault, and based on this, preventive measures are formulated. In a fault tree model, the occurrence of the top-level event may be determined by all the underlying events. However, in practice, the occurrence of the top-level event is often caused by a subset of the underlying events (cut sets). When the removal of any event in the cut set results in the non-occurrence of the top-level event, the cut set is referred to as a minimal cut set. The identification of minimal cut sets in the fault tree reveals various patterns of top-level event occurrences, highlighting the core role of minimal cut sets in fault tree analysis.

(2) Fault Diagnosis Expert System

A fault diagnosis expert system is an integrated system that combines artificial intelligence technology, aiming to simulate human experts in system fault data retrieval and diagnosis. These systems mainly achieve accurate and rapid diagnosis of complex system faults by integrating components such as specialized knowledge bases, reasoning mechanisms, and user interfaces. By combining expert knowledge with computer technology, a fault diagnosis expert system is constructed. This system follows specific reasoning algorithms and facilitates communication between the user and the computer through a human-computer interaction interface. The user needs to answer the questions posed by the system, and the system performs logical reasoning based on these questions and answers, ultimately deriving and presenting the diagnostic conclusion.

(3) Integration of Fault Tree Analysis and Fault Diagnosis Expert System

Through comparative analysis of the diagnostic expert system and fault tree analysis, commonalities between them can be identified. First, the fault tree can be seen as a fault model within the fault diagnosis expert system. The reasoning process of the expert system and the logical structure of the fault tree have similar designs. Additionally, from the perspective of knowledge acquisition, the fault tree represents a standardized knowledge framework. Using this framework to construct the knowledge base of the diagnostic expert system not only clearly outlines the solution path to the diagnostic problem but also significantly reduces the difficulty of knowledge acquisition for the system.

The knowledge base constructed based on fault tree analysis contains not only the rich experience and causal relationships between faults in NEVs but also preserves the unique hierarchical relationships between the conclusions in the fault tree. Transforming the fault tree into a knowledge structure that combines a knowledge graph framework with rules effectively supports reasoning and analysis in the fault diagnosis process. This method not only facilitates the acquisition and simplification of fault diagnosis knowledge but also improves the efficiency and accuracy of the diagnostic process, providing technical support for the fault data retrieval system in this research on NEVs.

3.1.2. The Logical Integration of Large Language Models and Knowledge Graphs.

Although LLMs have demonstrated excellent capabilities in natural language processing tasks, they exhibit limitations when faced with complex knowledge reasoning tasks that require deep and responsible reasoning. First, LLMs often cannot accurately answer specialized knowledge questions that go beyond their pre-trained content, especially those that rely on outdated knowledge or require long logical chains or multiple knowledge jumps. Second, the shortcomings of LLMs in responsibility, interpretability, and transparency increase the risk of generating erroneous information or harmful text. Finally, the training process of LLMs is not only costly and time-consuming but also presents a continuous challenge in keeping their knowledge up to date [34].

When using pure LLMs with chain-of-thought prompts to answer questions about the common faults of specific models of NEVs, the model typically fails to provide accurate answers beyond its pre-training phase. Additionally, it shows a lack of common sense reasoning and insufficient handling of semantic ambiguities. Given the limitations of relying solely on the model for reasoning, introducing fine-tuning training and KGs as external knowledge sources becomes a natural and promising solution aimed at enhancing the model's reasoning ability.

In Figure 2, the response using only the large language model is shown, while in the example in Figure 3, the model plays the role of “translation” + “reasoning.” It transforms the input question into machine-readable commands for the graph to perform search and reasoning. The complete knowledge graph successfully retrieved the correct knowledge, correcting the fault reasoning model that was fine-tuned with downstream learning on electric vehicle failure knowledge. Due to the limited and non-updatable knowledge base, it led to a lack of fault knowledge, but the model provided reasonable fault data retrieval results.

Question: My hybrid vehicle model is experiencing an EV power limitation issue. The car fails to start, the fuel door has no response, and the restart has no response. What's wrong?"

LLM-only

(Chain-of-Thought Prompt): Let's think step by step.
Response: If your vehicle exhibits the above situation, there are some possible causes and suggestions: battery failure, software/sensor errors, or vehicle not being in safe mode. The hybrid system may have detected high-voltage system faults. To diagnose the issue, the best approach is to use professional diagnostic tools, as it is recommended to check the fault codes with these tools and connect to the system service for maintenance. For safety, avoid self-repair of the electrical system.

Figure 2. Only using LLMs.

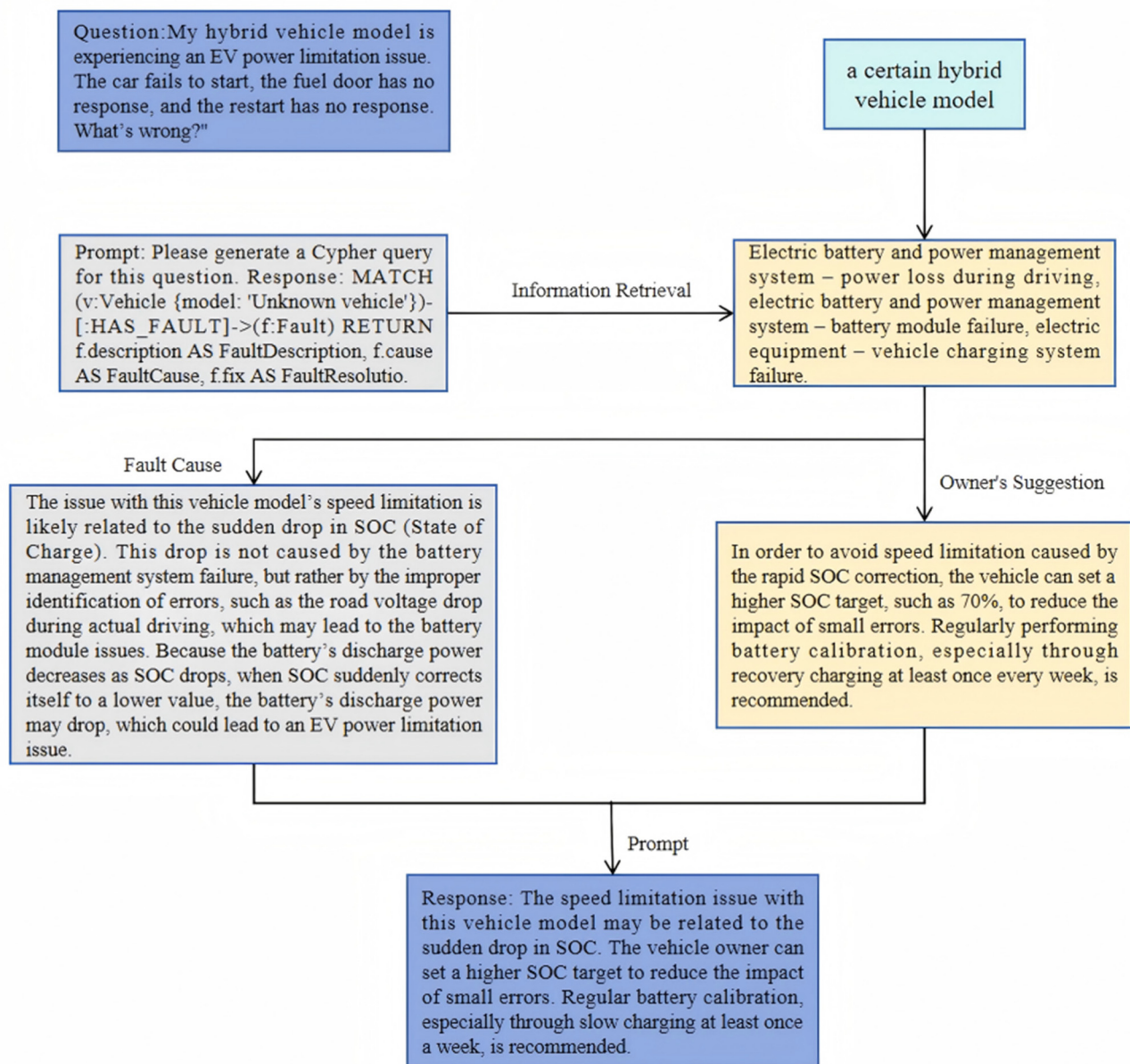


Figure 3. LLMs combined with knowledge graph retrieval.

3.2. The Framework Design of the Fault Data Retrieval System

The primary goal of developing an intelligent retrieval system for NEV fault data is to provide support for various users in aspects such as fault-related information, intelligent diagnostic analysis, fault repair and maintenance, equipment operation, and comprehensive knowledge services. Therefore, the overall design of the system requires a high level of fault information retrieval capability and sufficient knowledge capacity.

This study's intelligent retrieval system for NEV fault information uses a large language model as the reasoning engine and a knowledge graph as the knowledge base to analyze fault information and provide diagnostic and retrieval results. Its key features include leveraging the powerful reasoning and generation capabilities of LLMs, utilizing fine-tuning transfer learning techniques to handle and complete specific downstream tasks, and integrating the Neo4j database to store knowledge triples related to NEV faults.

Based on the fault knowledge model, the system provides fault information retrieval and analysis for users.

During the system's operation, when a vehicle experiences an abnormality, the user can input fault-related information through the front-end interface. After acquiring this information, the system processes it using a fault classification model to determine the fault type preliminarily. Depending on the user's specific situation, the system selects an appropriate query template and generates a corresponding Cypher query statement. It then performs query operations in the knowledge graph database, retrieves the results, and feeds them back to the fault reasoning and analysis model. The large language model performs fault reasoning on the fault information, determines the fault condition, and provides relevant suggestions, completing the entire fault data retrieval and diagnostic process.

The framework structure of the intelligent retrieval system for NEV faults data includes a large language model module, a knowledge graph module, an integrated reasoning module, and a human-computer interaction module, as shown in Figure 4.

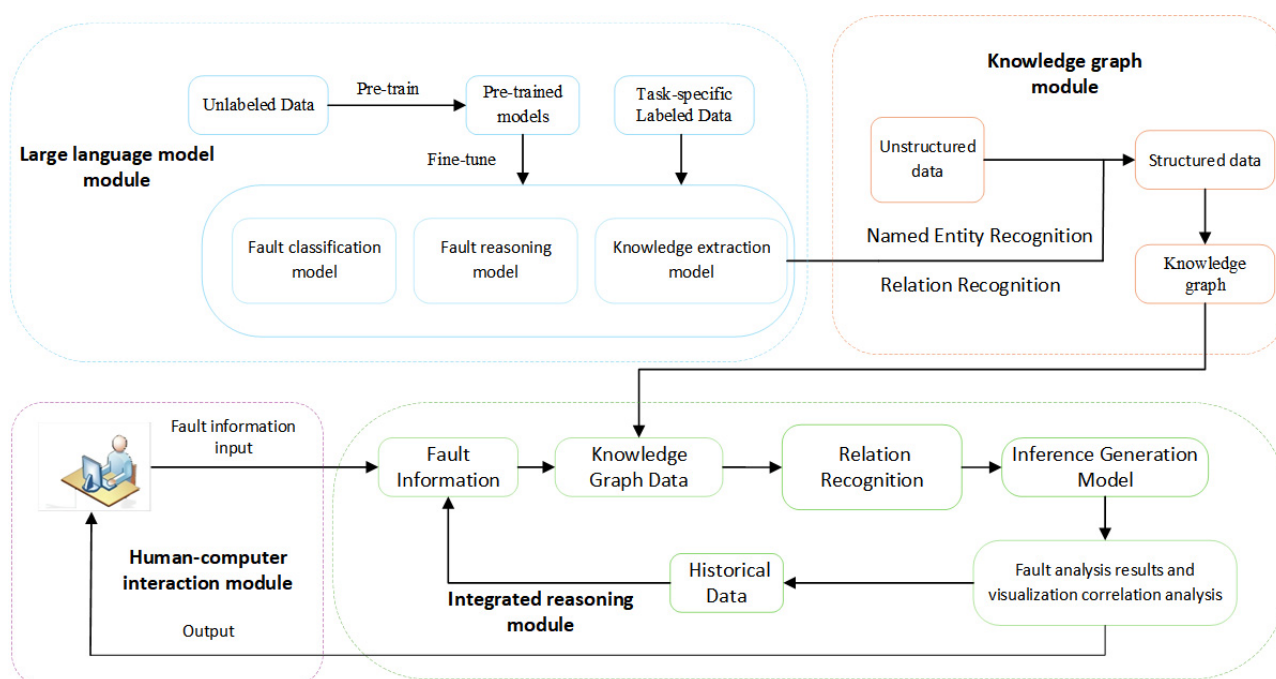


Figure 4. System design framework diagram. (The arrows indicate the direction of data flow between the modules.)

3.2.1. Large Language Models Module

The LLM module consists primarily of two parts: data processing and the training and fine-tuning of the LLM. This module serves as the foundation of the entire system. Fine-tuning is employed in the design to train the model with knowledge related to various fault issues in NEVs, which involves three main stages: data acquisition and preprocessing, model training, and training evaluation. The LLMs module is shown in Figure 5.

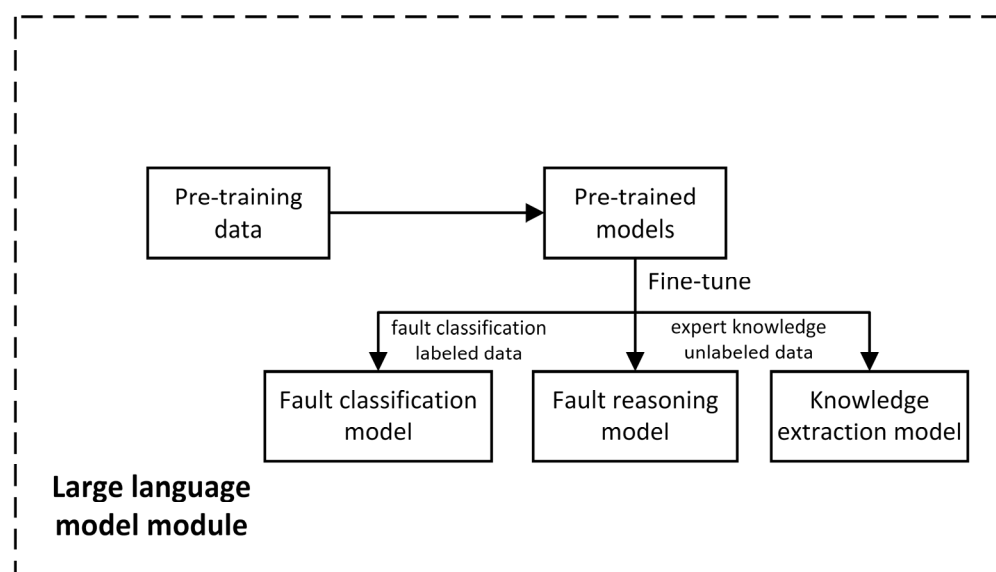


Figure 5. The LLM module. (The arrows indicate the direction of data flow between the modules.)

(1) Data acquisition and preprocessing

The primary data sources for this study include websites related to NEV faults, enterprise design, and production materials, as well as local and national standards (see Appendix A). A differentiated strategy is adopted for data acquisition: semi-structured data is collected using Python 3.12.2 and its associated toolkits, while unstructured data is obtained via text parsing and OCR recognition [35]. Due to differences in website structures and scanning limitations, the collected data may contain invalid or incorrectly formatted entries. Therefore, preprocessing is necessary, which includes data format conversion and filtering, primarily achieved using regular expressions and custom scripts. This ensures that the data is adequately prepared and valid, providing strong support for subsequent fault retrieval and analysis.

(2) Model training

The purpose of model training is to enable the model to learn domain-specific data deeply, allowing it to accurately identify and analyze fault information for NEVs. By comparing different training strategies, the best training process for handling NEV faults can be identified. In this study, DeepSpeed acceleration technology and CUDA GPU computation are used to improve training efficiency. During experiments, parameters such as learning rate, batch size, and training epochs are continually adjusted to optimize model performance. The training process leverages the Seq2SeqTrainer from the Transformers library for efficient model training. Once training is complete, the model is saved, and key performance metrics, such as loss variations and accuracy, are recorded for future evaluation and application.

(3) Model evaluation

The Model evaluation primarily focuses on assessing the system's performance in the intelligent retrieval of NEV fault information, particularly in expert Q&A tasks. The model is tested using existing Q&A datasets, and its performance is measured by calculating BLEU [36] and ROUGE [37] scores, which evaluate text similarity and semantic similarity between the model's responses and standard answers [38]. After considering these metrics, the model that performs best in terms of text similarity, semantic similarity, and logical coherence is selected as the final system fault knowledge model.

3.2.2. Knowledge Graphs Module

To establish an intelligent system that can truly solve the problem of NEV fault information retrieval, it is necessary to build a database of NEV faults and a relatively complete knowledge system. The realization of this function requires the knowledge body graph module. In the study, a combination of top-down and bottom-up methods was used to construct a knowledge graph of NEV fault information; on the basis of data preprocessing, the function was realized through knowledge modeling, knowledge extraction, and knowledge storage [39,40]. The framework of the knowledge graph module is shown in Figure 6.

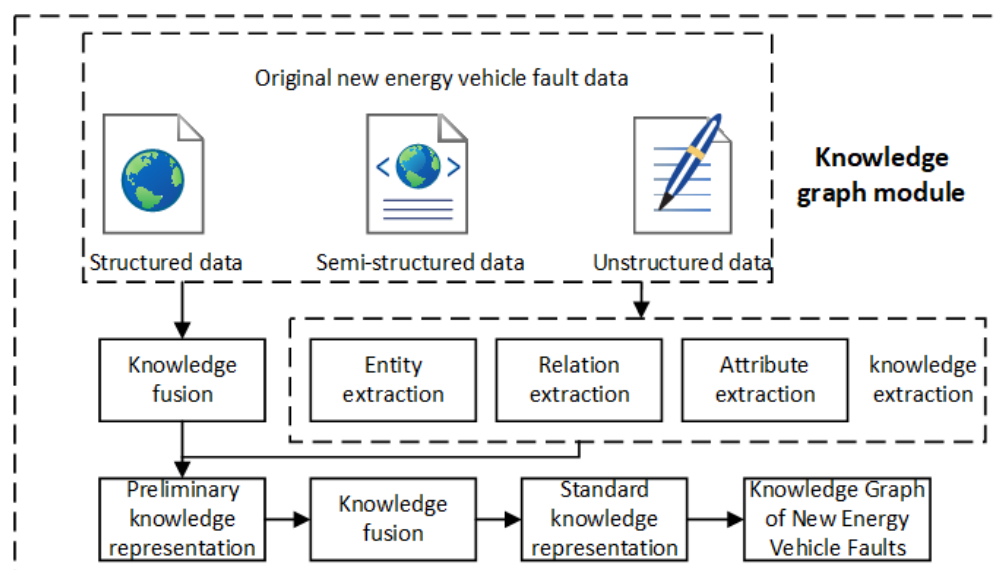


Figure 6. The knowledge graph module. (The arrows indicate the direction of data flow between the modules.)

(1) Knowledge modeling

This stage focuses on building the ontological framework structure for NEV fault issues. Through extensive research into real-world scenarios and data analysis, and based on the diagnostic characteristics of the fault tree model, the knowledge system for NEV faults is structured and layered. Relevant categories, attributes, and relationships are defined. Ontology modeling tools like Protégé are used [41], and an improved seven-step top-down method is applied to establish the ontology database for NEV faults.

(2) Knowledge extraction

This step involves extracting entities, relationships, and attributes from preprocessed data to construct and enrich the systematic knowledge graph for NEV faults. It primarily focuses on two types of triples: (entity, relationship, entity) and (entity, attribute, attribute value). To enhance the efficiency of knowledge extraction for fault information, extraction techniques are chosen based on the data structure. For semi-structured data, entities and attributes are extracted using rule-based scripts; for unstructured data, deep-learning models are employed to extract entity relationships [42,43].

(3) Knowledge storage

Currently, knowledge storage methods can be categorized into table-based and graph-based storage methods. Due to its strong expressiveness, ease of understanding, scalability, and ability to facilitate cross-domain knowledge integration, this study adopts a graph-based storage method using Neo4j [33]. This enables the storage, subsequent updates, and reasoning of knowledge related to NEVs.

3.2.3. Integrated Reasoning Module

The core function of the integrated reasoning module is to correctly interpret user intent, classify faults using a fault classification model, retrieve related fault data from the knowledge graph database, and use the LLM for centralized reasoning. This enables intelligent fault data retrieval and provides actionable recommendations. The system adopts the Model-View-Controller (MVC) architecture and is developed using Python's Flask framework. It includes the data layer, algorithm layer, business layer, and presentation layer. Various tools, including Flask, Python 3.12.2, Neo4j, MySQL, ECharts, PyTorch 2.2.0, and Transformers 4.38.1, are employed in its design and implementation. This module consists of three main parts: fault classification, information retrieval, and model reasoning. The integrated reasoning module is shown in Figure 7.

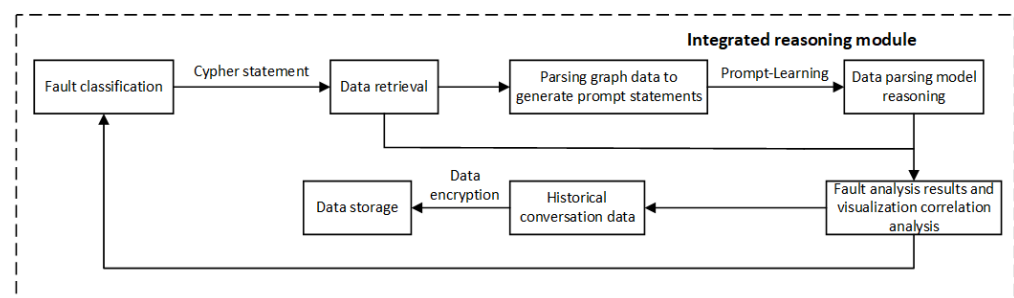


Figure 7. The integrated reasoning module. (The arrows indicate the direction of data flow between the modules.)

(1) Fault classification

The fault classification module is designed to interpret user intent and preliminarily determine fault categories based on the information provided, laying the foundation for subsequent information filtering. This relies on fault classification models and focuses on mainstream classification techniques such as deep learning, machine learning, and pre-trained Transformer-based models [44]. Fault data for NEVs is annotated and applied to various classification models; the performance of these models is compared to select the one that achieves the best classification performance.

(2) Information retrieval

This module converts the fault information output by the fault classification model into specific Cypher query statements, which are then executed on the Neo4j knowledge graph database. For example, to query the relationship between a specific brand and its NEV electronic control system faults, the system executes the following query: “MATCH p = (: Complaint Brand{Complaint Brand:' BYD'})-[rel*1..2]->(n) RETURN p, rel, n” This query is encapsulated into a function and placed in the data layer for invocation. The retrieved data is transformed into a predefined format for prompt statements, which are then submitted to the knowledge model for NEV faults for comprehensive judgment. The key to system design lies in mastering Neo4j's Cypher query language and understanding the Prompt format requirements of LLMs to ensure the correct generation of query prompts. Finally, the results are outputted through the LLM.

(3) Fault reasoning

This module uses the interactive API of the LLM to process the retrieved Prompt information and submit it to the fault knowledge model trained with expert knowledge. It identifies the causes of faults in NEVs and provides corresponding recommendations. Simultaneously, the retrieved data is presented in the form of graph animations, visualizing the knowledge graph and the entire reasoning chain.

4. Results

The fundamental requirement for constructing an intelligent retrieval system tailored for NEV fault data is to accumulate sufficient vehicle fault data and knowledge including related concepts, methodologies, instrument tools, logic, and information resources related to dealing with fault problems. The principal challenge encountered in practical application is how to obtain a sufficient quantity of pertinent data and knowledge resources, followed by their systematic organization and amalgamation to form a constantly enriched, holistic, and comprehensive knowledge architecture.

4.1. Data Acquisition and Preprocessing

The data sources mainly include text, audio, video, and other digital resources on the internet. In the study, we accessed various available resources, websites, and online databases related to automobiles through manual statistics and web crawler technology. After analyzing the structural features and data feedback methods of various websites, we systematically merged and integrated NEV data, which served as the basis for constructing the NEV knowledge graph and intelligent fault data retrieval system.

The dataset pertaining to NEVs encompasses three primary categories: battery electric vehicles, plug-in hybrid electric vehicles, and range-extended electric vehicles. This comprehensive dataset comprises essential details regarding the specific specifications and technological types of existing NEVs. A cumulative total of 822 distinct vehicle models has been collated, which includes 631 battery electric vehicle models, 35 range-extended electric vehicle models, and 171 plug-in hybrid electric vehicle models. Moreover, an extensive compilation of 25,000 fault types has been compiled into 12 principal classifications, with further subdivision into 122 subcategories based on diverse fault characteristics. Additionally, an impressive repository of 22,709 instances of expert knowledge has been meticulously sorted out under various fault scenarios, providing a robust foundation for the preprocessing of data and the generation of knowledge for LLMs.

In the application of LLMs, it is imperative to undergo both training and fine-tuning phases, particularly in the context of QA-type applications, wherein the objective is to facilitate effective question-answering. The process of fine-tuning is systematically executed in accordance with the following steps:

Step 1: Data sanitization, which entails the meticulous purification of the original dataset by eliminating special characters, punctuation, HTML tags, etc., thereby guaranteeing the integrity and uniformity of the information.

Step 2: Data format conversion is the transformation of the sanitized data into a JSON format that is conducive to model ingestion, typically structured in an interactive question-answer paradigm. Each data sample comprises a query and a corresponding response, delineated by specific delimiters. An example of the data format conversion is as follows: question, {"conversations": [{"role": "user", "content": "Throttle Position Sensor, Structure and Working Principle."}, {"role": "assistant", "content": "Greetings! The throttle position is situated posterior to the hose and antecedent to the manifold. The throttle constitutes a regulatable valve that governs the inflow of air into the engine. Subsequent to the air's entry into the intake manifold, it is combined with gasoline to create a flammable mixture, which is thereafter ignited to generate power."}]}

Step 3: Insertion of demarcation tokens, to apprise the model when it should start processing the problem and the corresponding answer. Special tokens, including [gMASK], sop, <|system|>, <|assistant|>, and others, are interspersed at the commencement, conclusion, and specific junctures within the input text, thereby facilitating the model's accurate comprehension of the input data.

Step 4: Encoding of data, entailing the conversion of the annotated text into a recognizable code format for the model. This process employs word embedding techniques to

transform textual data into vector representations and supplements them with positional encoding to constitute the model's input vector.

Following these preprocessing procedures, the primal dataset is meticulously converted into an appropriate format conducive to the fine-tuning of LLMs, subsequently undergoing meticulous annotation and encoding processes [45,46]. This measure can significantly enhance the efficacy of LLMs when applied to the downstream task of expert knowledge acquisition for NEVs.

4.2. New Energy Vehicles Fault Model Training

4.2.1. New Energy Vehicle Fault Knowledge Model Training

The core objective of fine-tuning LLMs is to achieve efficient adaptation of domain knowledge with limited resources. Traditional full-parameter fine-tuning requires updating all model weights, which is computationally expensive. To address this, parameter-efficient fine-tuning methods have emerged, which update only a subset of parameters or optimize prompts, significantly reducing resource consumption while maintaining model performance. This study focuses on two cutting-edge parameter-efficient fine-tuning methods—LoRA (Low-Rank Adaptation) and P-Tuning V2—to investigate which method is more suitable for training on electric vehicle fault data.

(1) P-Tuning V2 fine-tuning

P-Tuning V2 is an emerging fine-tuning strategy designed based on traditional text-based prompting, which, although effective in some cases, has limited expressive power due to its reliance on a fixed vocabulary and is unable to fully exploit the model's potential expressive space. Therefore, it enhances the model's performance in fault data retrieval by introducing learnable continuous vectors (referred to as "prompts" or "soft prompts") without directly modifying the pre-trained model's parameters. These continuous vectors are designed to be embedded into the model's input as task instructions, guiding the model to generate outputs for specific tasks. P-Tuning V2 optimizes these soft prompts rather than the model's weights, enabling adaptation to expert knowledge of NEV faults while keeping the pre-trained parameters unchanged. This approach is particularly suitable for scenarios with large-scale models in the current system, where directly fine-tuning the model would be computationally expensive [47].

(2) LoRA fine-tuning

LoRA is a fine-tuning method designed for transformer architecture. It achieves fine-tuning by introducing additional, low-rank adaptation matrices into the model's self-attention and feed-forward networks [48]. The dimensions of these matrices are relatively small, significantly reducing the number of parameters that need to be updated during fine-tuning. The key idea behind LoRA is to capture task-specific variations through low-rank matrices, rather than directly modifying the original model's weights. This method reduces the number of parameter updates during fine-tuning while maintaining high performance across various tasks.

(3) Experiment and results analysis

This study uses loss and accuracy in model training to predict and evaluate the performance of the model. Loss is calculated during the model training phase and is used to assess the difference between the model's predictions and the actual labels. The cross-entropy loss function is used to compute the difference between the predicted probability distribution at each time step and the actual labels. At the same time, BLEU and ROUGE scores are used to evaluate the quality of the generated text, with these scores calculated by comparing the generated text to the reference text, providing a comprehensive measure of the accuracy and fluency of the model's generated text.

In this experiment, data from 22,709 expert Q&A pairs were processed, filtering out question-answer pairs with a maximum output length of 256 tokens to ensure proper training for the LLM. The data were then split into training and testing sets at an 80/20 ratio, resulting in 18,107 training samples and 4408 testing samples.

The model training experimental results are shown in Figure 8.

From the experimental data, it can be observed that Figure 8a presents the loss value trend of the model using LoRA technology, while Figure 8b illustrates the loss value trend of the model using P-Tuning V2 technology. A comparison between the two shows that the LoRA training method results in a faster decline in loss value during the training process. This indicates that the LoRA training method adapts more quickly to the NEVs fault expert knowledge training data, learning the complex relationships between various fault information. This rapid decline in loss value is evidence of good model training performance, suggesting that the model demonstrates high efficiency and accuracy in recognizing and understanding the training data.

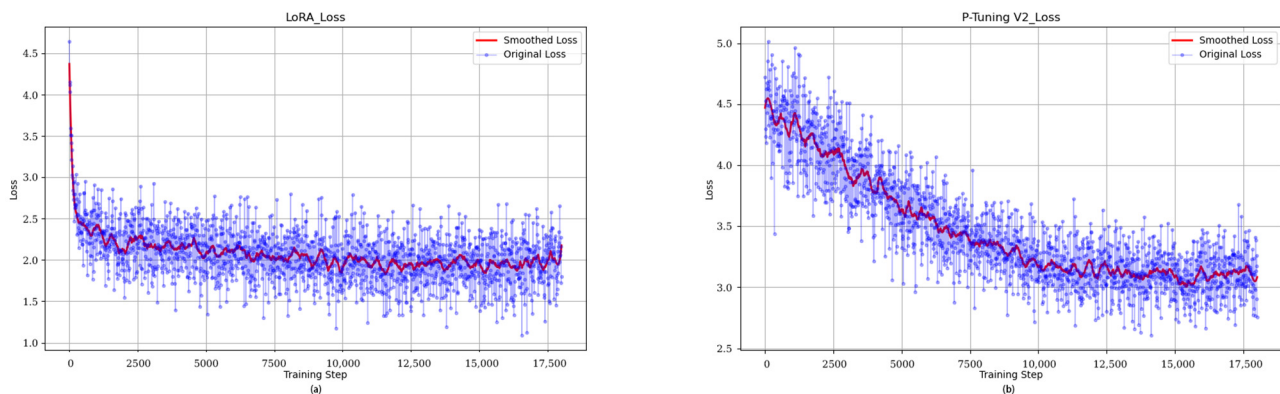


Figure 8. The training loss curve chart of the LoRA and P-Tuning V2 models.:**(a)** LoRA model training loss; **(b)** P-Tuning V2 model training loss.

In the model evaluation phase, the BLEU-4 and ROUGE-L scores are shown in Figure 9, with the BLEU-4 score on the left and the ROUGE-L score on the right.

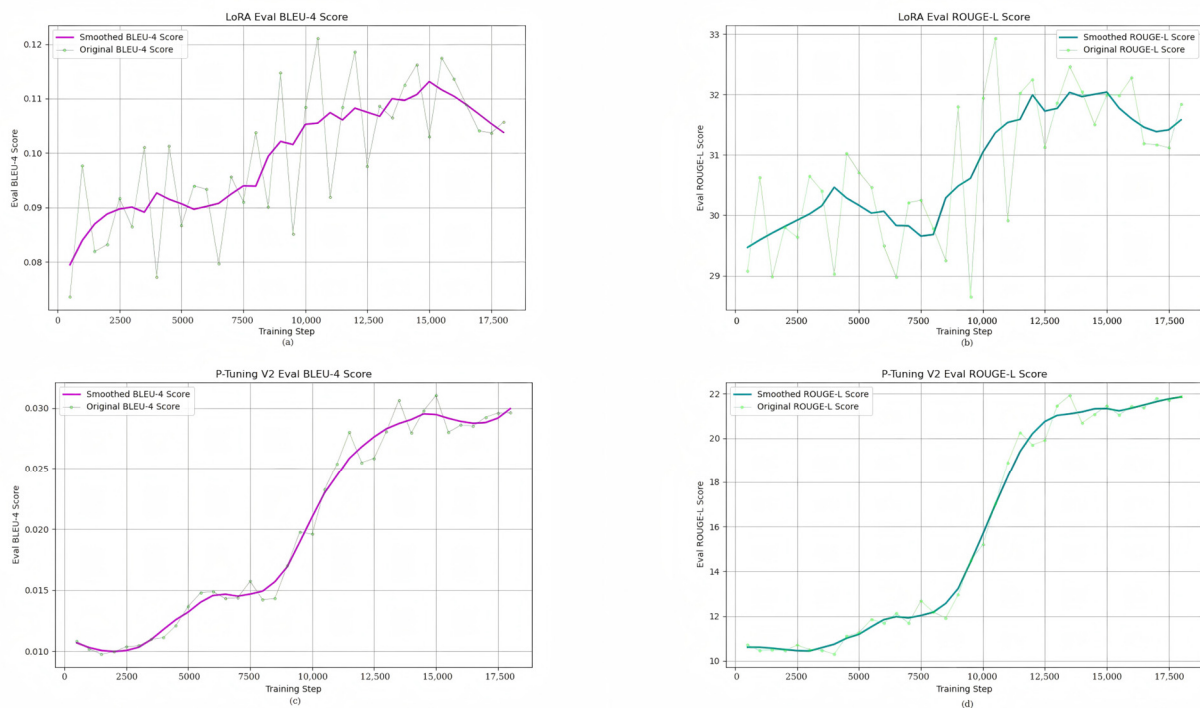


Figure 9. BLEU-4 and ROUGE-L scores for LoRA and P-Tuning V2 models during evaluation: (a) BLEU-4 score of the LoRA model; (b) ROUGE-L score of the LoRA model; (c) BLEU-4 score of the P-Tuning V2 model; (d) ROUGE-L score of the P-Tuning V2 model.

Figure 9 presents the evaluation results of models fine-tuned using LoRA and P-Tuning V2 on the NEVs fault expert Q&A generation task, with BLEU-4 and ROUGE-L scores as the primary evaluation metrics. This figure consists of four subplots, each illustrating the performance of the fine-tuned models in terms of text generation quality, relevance, and consistency. (a) LoRA Eval BLEU-4 Score. This subplot shows the BLEU-4 score trend for the LoRA fine-tuned model. The original score (green line) fluctuates, while the smoothed score (purple line) increases overall, indicating improved text generation quality. (b) LoRA Eval ROUGE-L Score. This subplot presents the ROUGE-L score trend for LoRA. The original score (green line) shows variability, whereas the smoothed score (blue line) rises, suggesting enhanced text relevance and coherence. (c) P-Tuning V2 Eval BLEU-4 Score. This subplot illustrates the BLEU-4 score for P-Tuning V2. Similar to (a), the original score fluctuates, while the smoothed score steadily increases, reflecting improved text generation accuracy. (d) P-Tuning V2 Eval ROUGE-L Score. This subplot depicts the ROUGE-L score trend for P-Tuning V2. The original score has noticeable fluctuations, while the smoothed score gradually rises, indicating improved text consistency.

The comprehensive experimental results show that both P-Tuning V2 and LoRA fine-tuning methods significantly improve the performance of the base model in the NEVs fault expert Q&A generation task. Specifically, P-Tuning V2 has a slight advantage in model input tokens, allowing it to train longer token sequences. On the other hand, LoRA performs better on large-scale datasets, demonstrating faster loss reduction, and its question-answering accuracy in BLEU-4, ROUGE-1, ROUGE-2, and ROUGE-L scores generally outperforms P-Tuning V2. The low-rank matrix update strategy of LoRA offers clear advantages in parameter efficiency and computational cost. Therefore, this study selects the LoRA fine-tuned model as the LLM for fault data retrieval.

After merging the optimized model weights obtained from training with the original pre-trained model weights, a complete model file was saved. After configuring the

tokenizer for ChatGLM3-6B, the model was transferred to the GPU and switched to evaluation mode for dialogue assessment. At this stage, Moreover, it demonstrated a good grasp of the additional NEV fault expert knowledge introduced during training while maintaining its logical reasoning and analytical capabilities. It was observed that the trained model effectively avoided hallucinations even without extra prompts or external knowledge bases as supplementary data sources. This makes it well-suited to serve as the LLM for NEV fault data retrieval in this study.

4.2.2. New Energy Vehicle Fault Classification Model Training

Since annotated data is sufficient in the field of fault classification, and BERT models often achieve better results in NLP tasks, considering the resource consumption of model deployment, we conduct a fault text classification experiment using Long Short-Term Memory (LSTM) and BERT. Additionally, TF-IDF + Naïve Bayes and TF-IDF + Ridge Regression classifiers are used as baseline comparison groups.

The study employs a cross-validation method with training and test sets, dividing the NEVs fault corpus into 10 parts, with 8 parts used for training and 2 parts for testing. The dataset is not shuffled. The deep-learning Transformers library provided by Hugging Face is used, and the pre-trained Chinese hfl/chinese-roberta-wwm-ext model from iFlytek is loaded for training. In the BERT-based text classification experiment, the tokenization and preprocessing step sets the maximum sequence length to 512 tokens. The experimental model includes a DistilBertModel as the pre-trained model, followed by a fully connected layer (768 to 768), a ReLU activation function, a Dropout of 0.2, and a final fully connected layer (768 to 12) for classification. The classifier layer parameters of the pre-trained model are used to initialize the final fully connected layer. The optimizer is set to AdamW with a learning rate of 2×10^{-5} . Dynamic padding is used to ensure uniform data length within each batch. During training and testing, data shuffling is enabled for training, and the last incomplete batch is discarded. In the LSTM classification experiment, the dataset was tokenized and word2vec was used to train word embeddings, converting each question into a word vector matrix. The experiment was implemented using TensorFlow, with an optimized learning rate of 0.015, a batch size of 10, the Adam optimizer, and the Sigmoid function for final classification. The classifier's input was derived from the LSTM hidden layer output.

When evaluating the performance of the trained models, this study uses accuracy, recall, and F1-score as the main evaluation metrics. The calculation formulas are as follows:

$$P = \frac{TP}{TP + FP} \times 100\% \quad (1)$$

$$R = \frac{TP}{TP + FN} \times 100\% \quad (2)$$

$$F_1 = \frac{2 \times P \times R}{P + R} \times 100\% \quad (3)$$

TP (true positive) refers to the number of instances correctly predicted as positive, while FP (false positive) represents the number of instances incorrectly predicted as positive when they are actually negative. Similarly, FN (false negative) denotes the number of instances incorrectly predicted as negative when they are actually positive. These metrics are used to evaluate the classification performance of the model. P (precision) represents the accuracy of the model's predictions, R (recall) reflects the model's ability to retrieve relevant instances, and the F1-score, as the harmonic mean of precision and recall, provides a comprehensive assessment of the model's overall performance. To ensure the

reliability of our experimental results, we calculated 95% confidence intervals for precision, recall, and F1 Score. The results indicate that the precision ranges between 72.02% and 84.59%, recall between 74.25% and 85.61%, and F1 Score between 73.13% and 85.13%. These confidence intervals confirm that the model's performance is stable and not due to random chance. The results of some fault classification algorithm experiments are shown in Table 3.

Table 3. The results of some fault classification algorithm experiments.

Method	Category	Accuracy P%	Recall Rate R%	F1 Score %
BERT	Braking System Fault	87.22	88.36	88.03
	Electric Drive System Fault	85.68	86.24	85.96
LSTM	Braking System Fault	81.14	82.59	81.86
	Electric Drive System Fault	84.27	85.61	84.93
TF-IDF + Naïve Bayes	Braking System Fault	71.42	72.79	72.10
	Electric Drive System Fault	66.34	71.66	68.90
TF-IDF + Ridge Regression	Braking System Fault	73.19	72.59	72.89
	Electric Drive System Fault	77.15	79.60	78.36

From the analysis of the above experimental results, it can be concluded that Naïve Bayes, due to its simplicity and efficiency, is usually suitable as a baseline model, while Ridge Regression performs better when dealing with fault data classification, especially in datasets with high feature correlation. Ultimately, the BERT model significantly outperforms both the deep-learning LSTM model and the machine learning approaches, including TF-IDF + Naïve Bayes and TF-IDF + Ridge Regression, in classifying NEVs faults.

The BERT model utilizes the fully bidirectional attention mechanism of the Transformer to outperform the LSTM model in processing fault information efficiently and capturing contextual details comprehensively. While LSTM's bidirectional configuration can handle contextual information, its sequential processing method may cause information loss in long texts. BERT optimizes the evaluation of the importance of different parts of an input sentence through its self-attention mechanism, enabling superior performance in handling long-distance dependencies and complex sentence structures. In the task of NEVs fault classification, BERT's fully bidirectional contextual understanding, strong generalization ability, and the advantage of reducing task-specific engineering requirements make it the preferred model for processing subtle textual differences. Based on the characteristics of the NEVs fault corpus and experimental results, this study selects the BERT model as the fault classification model for the NEVs fault data retrieval system.

4.3. Knowledge Extraction

4.3.1. Named Entity Extraction in the Domain of NEV Faults

Entity extraction is a crucial step in constructing knowledge graphs, with its core task being the preprocessing of unstructured text data and the use of named entity recognition techniques to identify entities related to new energy vehicle faults. Given the diversity, specialization, and complexity of new energy vehicle fault knowledge, this study utilizes

the BERT-BiGRU-CRF model for named entity extraction in this domain. Furthermore, to further optimize entity recognition performance for specialized terms such as equipment names, fault types, and vehicle models, this study also compares the proposed approach with the ChatGLM3-6B model, which utilizes In-Context Learning technology.

In this study, the performance of different models in the task of named entity recognition for new energy vehicle faults is compared. The evaluation is conducted using precision (P), recall (R), and F1-score as key metrics to assess and compare the effectiveness of entity extraction across models. Specifically, both the In-Context Learning model and the BERT-BiGRU-CRF model are evaluated on a self-constructed test set of new energy vehicle faults, with their ability to identify relevant entities in this domain serving as the basis for comparison.

A comprehensive analysis indicates that the In-Context Learning-based ChatGLM3-6B model demonstrates strong adaptability in the task of named entity extraction for new energy vehicle faults. The results of named entity recognition are shown in Table 4.

Table 4. Model Named Entity Recognition Performance.

Model Name	Entity Category	Accuracy P%	Recall Rate R%	F1 Score %
BERT-BiGRU-CRF	NEV Brand	78.33	66.29	71.80
	NEV Model	76.70	70.33	73.38
	NEV Fault Category	63.48	51.29	56.74
	NEV Fault Name	80.69	67.72	73.64
	NEV Fault Phenomenon	52.96	31.89	39.81
ChatGLM3-6B In-Context Learning	NEV Brand	88.64	63.28	73.84
	NEV Model	81.18	68.45	74.27
	NEV Fault Category	75.32	62.13	68.09
	NEV Fault Name	89.65	65.56	75.74
	NEV Fault Phenomenon	65.74	59.12	62.25

4.3.2. Knowledge Reuse and Substitution in the Domain of NEV Faults

The previous work has completed the named entity recognition in the field of NEV faults, laying the foundation for the construction of a knowledge graph and providing precise semantic analysis and user intent recognition. However, due to factors such as geography and culture, users' expressions of the same thing can be diverse, leading to multiple lexical descriptions of the same item in language. For example, different platforms may use different naming conventions for the same vehicle model, such as "Qin PLUS," "Qin PLUS DM-i" and "Qin PLUS EV," which refer to the same model. Therefore, the system needs to expand the knowledge base to accommodate different user understandings. If the issue of lexical diversity is not addressed, the system may fail to accurately identify entities during knowledge retrieval, resulting in erroneous information retrieval and misjudgment of fault data.

To address the challenge of handling multiple expressions in user queries where the system struggles to locate answers in the knowledge graph, this study proposes a knowledge-based construction method. This method aims to encompass technical terms and their numerous expressions. By extensively collecting professional vocabulary related to NEV faults and their various expressions, these different expressions are incorporated as entities into the knowledge base. At the same time, clear corresponding relationships are established for these entities, allowing them to be linked to relevant technical terms so

that users can find the corresponding fault entities and relationships through rephrased terms during retrieval.

Through reviewing the literature on NEV faults and consulting with repair design professionals, a total of 217 NEV fault terms with multiple meanings have been compiled, as detailed in Table 5.

Table 5. NEV Fault Terminology Paraphrase Table.

ID	Domain Term	Synonyms
001	Battery Cell Insulation	Battery Foam Padding
002	Motor Cover	Front Cover
003	Motor Rear Cover	End Cap, Transparent Cap
004	Qin PLUS	Qin PLUS DM-i, Qin PLUS EV
...	...	...
217	Volvo XC60 New Energy	Volvo XC60 RECHARGE

4.4. System Development Technology

The evolution of an intelligent search system tailored for fault diagnosis in NEVs has adopted a diverse array of development methodologies across multiple modules and hierarchical levels, as illustrated in Figure 10. On the database level, the Cypher query language is employed to execute read and write operations on the NoSQL database, Neo4j. Regarding the programming languages utilized in system development, Python, HTML, and CSS have been chosen for their efficacy. On the algorithmic design level, Python’s robust data analysis libraries—NumPy, Pillow, and Matplotlib—are harnessed to facilitate model training, evaluation, and data processing. For the selection of deep-learning models, the PyTorch framework is adopted, with the transformer serving as the model repository. In the domain of Web development, ECharts is utilized as the front-end data visualization framework, while Flask provides the structural foundation for the development environment.

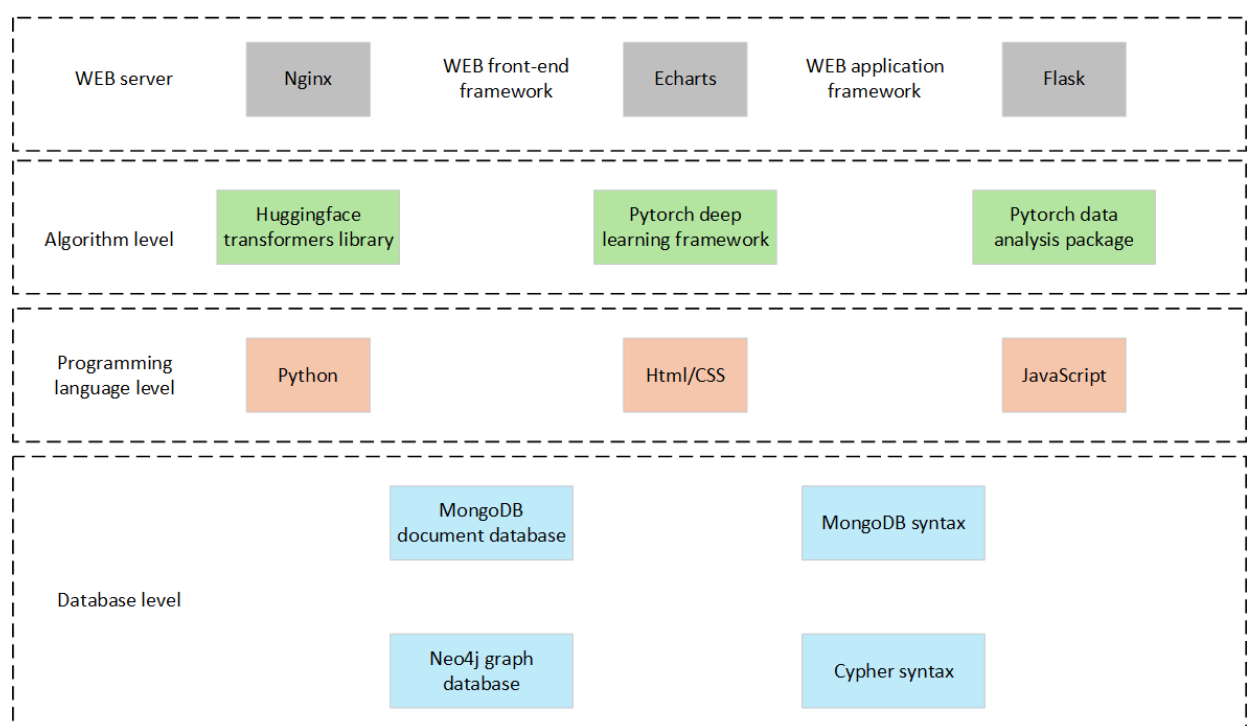


Figure 10. System development technical implementation.

4.5. System Demonstration and Testing

4.5.1. System Presentation

The retrieval mechanism embedded within the intelligent retrieval system tailored for fault data analysis of NEVs fundamentally encompasses two pivotal stages at its core: Initially, a sophisticated classification algorithm is employed to conduct a detailed categorization of NEV malfunctions. Subsequently, advanced text analytics are utilized to distill critical information, such as the vehicle model and its series. Leveraging this information, the system diligently retrieves corresponding fault data and provides it as a Prompt to the reasoning model that incorporates expert knowledge and scavenges the knowledge graph for pertinent fault information, which is then fed into a reasoning model imbued with expert knowledge to generate a comprehensive fault data retrieval prompt. Ultimately, the system will progressively generate and display the entire fault data retrieval results and the knowledge graph visualization data.

Figure 11 illustrates the interactive interface of the NEV fault data retrieval system, where users can enter fault descriptions or queries in the input box and click send to receive real-time streamed fault analysis results. For continuous conversations, users can opt to allow the large language model to retain contextual information, enabling it to reference previous analysis data for further reasoning. If no relevant information is found in the database, the large language model will independently conduct reasoning and provide supplementary suggestions, catering to various fault retrieval needs while offering a more intelligent and convenient fault diagnosis service.

Additionally, the system integrates a visualized fault data retrieval interface, enhancing correlation analysis and intuitively displaying the reasoning chain. For instance, when a user asks, “My BYD Qin experienced serious degradation. Can you help determine the specific fault?” the system returns an analysis result (as shown in Figure 11, where blue represents searching for vehicle models, green represents similar vehicle models, yellow represents complaint type, and red represents complaint content), indicating that battery degradation may be due to the end of its lifespan, power battery and energy management system faults could result from multiple battery cell failures or circuit issues, and battery undercharge might be caused by insufficient charging, leading to vehicle malfunction. These results are generated by the large language model in conjunction with the knowledge graph, where the system identifies common faults for this vehicle model, performs logical reasoning based on real fault symptoms, and provides precise analysis and targeted recommendations, ensuring an efficient and intuitive fault diagnosis experience.

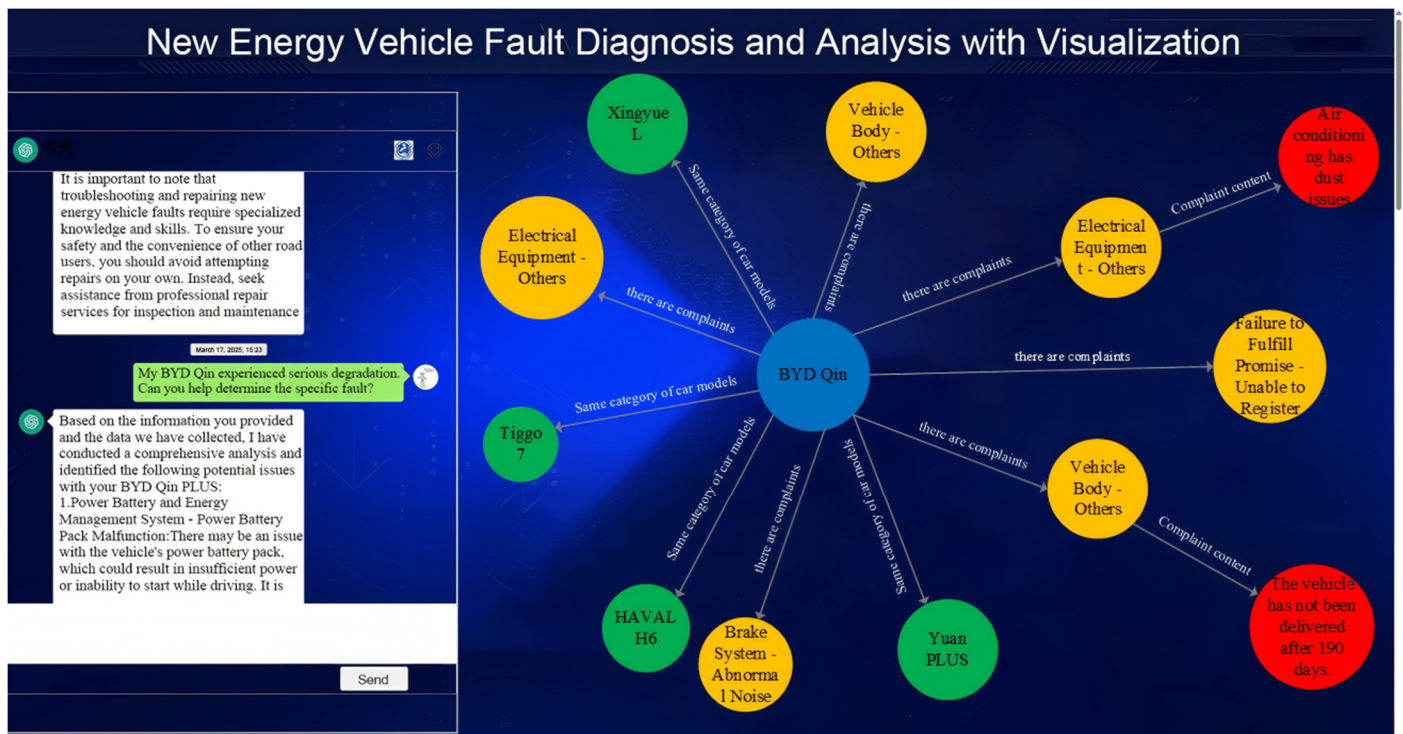


Figure 11. Fault diagnosis and analysis with visualization.

4.5.2. System Testing

To ascertain the efficacy and operational efficiency of the system, a series of functional assessments and practical applicability evaluations were undertaken. Refer to Table 6 for a detailed breakdown of the testing conditions.

Table 6. System testing environment.

Environment	Parameter	Environment	Parameter
Operating System	Windows11 64-bit	Memory	32 GB
CPU	Intel(R) Core (TM) i9-11900H CPU (Santa Clara, CA, USA)@ 2.50 GHz (2502 MHz)	Graphics card	NVIDIA GeForce RTX3070 Laptop GPU 8G GDDR6

Throughout the system functionality testing phase, the primary focus is placed on whether the system developed can respond to the user's operations and fault information, normally ascertaining the system's capability to adeptly respond to user inputs and to effectively relay fault notifications. Furthermore, it is crucial to evaluate the user's ability to interact with the system seamlessly, leveraging the intuitive nature of the interface display. The detailed procedural steps and corresponding outcomes are delineated in Table 7.

Table 7. System function testing.

Test Items	Operating Procedures	Expected Results	Test Results
User Interface Responsiveness Testing	Accessing the system interface via WEB pages	The system icons, buttons, and other user interface elements are displayed normally, and the user interaction functions are free of obstacles	Pass

Type of fault identification	Inputting fault information into the fault classification model in the background	The fault information inputted can be correctly received and parsed by the fault classification model.	Pass
Knowledge base search	The data parsed is retrieved from the knowledge base	The knowledge base can successfully retrieve data related to the parsed information	Pass
Fault data retrieval and graph visualization function	Submit the fault information after inputting it	The page can stream and analyze the data and display dynamic visual effects of the fault information-related graphs	Pass

The assessment entails the utilization of a comparative analytical framework, comparing the performance of mainstream search engines and similar popular Question Answering Systems at home and abroad. Owing to the void in the intelligent search domain for specialized NEVs faults, there exists an absence of a mature application system; this research represents an exploratory endeavor, with the comparative entities primarily comprising widely used intelligent question-answering systems such as Microsoft's Copilot, GPT-3.5 Turbo, Claude 3 Haiku, and the ERNIE Bot 4.0.

Throughout the assessment process, an emphasis is placed on scrutinizing the performance metrics of each system, including question-answer relevance, accuracy, textual logic, coherence, and response latency, as per queries crafted by automotive fault specialists. The assessment of question-answer relevance primarily gauges the pertinence and expertise of the responses proffered by the system in relation to the posited inquiries; precision evaluation pertains to the degree of alignment between the responses from the system and those from the automotive fault experts. To mitigate the bias inherent in comparing the efficacy of a particular knowledge model subsequent to targeted learning against its performance in an untrained state, the system assessment incorporates additional evaluation metrics, including textual logic, coherence, and response latency, among others. These metrics serve to evaluate the logical consistency and coherence of the generated outputs, with the final response latency being quantified as the average generation time per token, where a shorter latency equates to a higher score. The scoring mechanism is predicated on the automated scoring capabilities of GPT-4 and the percentile system of artificial assessment.

The findings indicate that the intelligent detection system for NEV faults, developed as part of this study, is similar to LLMs in terms of targeted responses, has a slightly higher accuracy than all other systems, exhibits a deficiency in both coherence and logical progress, and its response latency marginally surpasses that of its counterparts (refer to Figure 12).

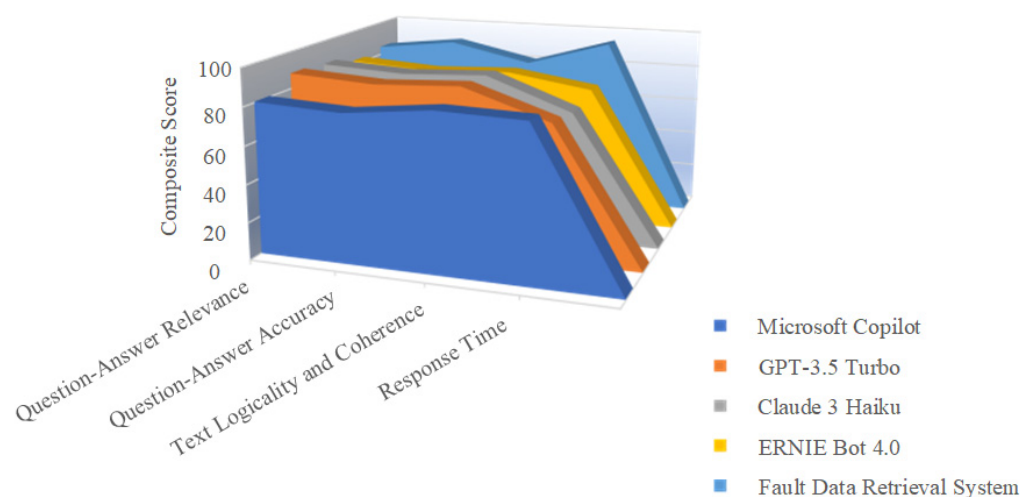
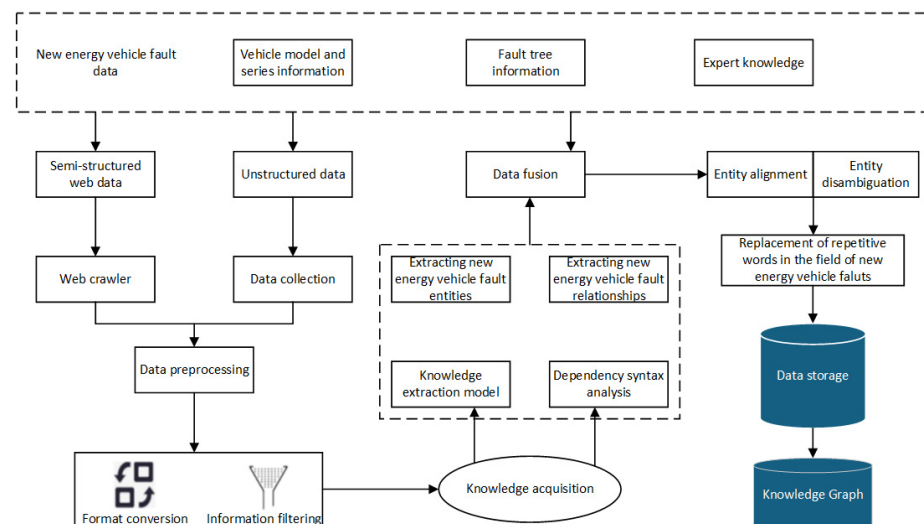


Figure 12. System usability assessment.

5. Discussion

5.1. Knowledge Graph and Fine-Tuning of Large Language Models

Through an in-depth analysis of the fault data procured from NEVs, a comprehensive knowledge graph pertaining to the failures encountered in NEVs has been meticulously designed and instantiated, leveraging the distinctive attributes of the automotive fault tree analysis methodology. The knowledge graph was systematically populated with specific fault categories pertaining to discrete vehicle models, encompassing 122 fault types along with their associated fault phenomena and etiologies. For LLMs, the ChatGLM3–6 model, engineered by Tsinghua University [49], was prudently chosen for the purposes of fault information retrieval and knowledge generation, in consonance with the unique characteristics of the problem at hand. An exhaustive extraction of fault information and knowledge relationships was extracted, culminating in the formulation of a comprehensive summary lexicon for the principal fault domains. Figure 13 delineates the procedural framework for constructing the fault knowledge graph tailored for NEVs.

**Figure 13.** Fault Knowledge Graph Construction Process. (The arrows indicate the direction of data flow between the modules.)

The training and fine-tuning of LLMs are indispensable in constructing sophisticated domain-specific intelligent information retrieval and question-answering systems. In the context of this investigation, the P-Tuning and LoRA fine-tuning techniques were selected for the fine-tuning and training of the ChatGLM3-6B model. Extensive experimental evaluations have demonstrated that both the P-Tuning V2 and LoRA fine-tuning methodologies considerably enhance the performance of the ChatGLM3-6B model when answering questions related to NEV malfunctions. In particular, P-Tuning V2 has an advantage in processing input tokens, enabling the concurrent training of longer token sequences. On the other hand, LoRA demonstrates superior efficacy on extensive datasets, showcasing enhanced accuracy metrics (Accuracy) in BLEU-4, ROUGE-1, ROUGE-2, and ROUGE-L for question-answering tasks. Its low-rank matrix updating approach is particularly advantageous in terms of parameter efficiency and computational expense. Owing to the intricate nature of the model calculations involved, a detailed analysis is not provided here. Figure 14 illustrates a comparative analysis of the output outcomes before and after the training of the LLM.

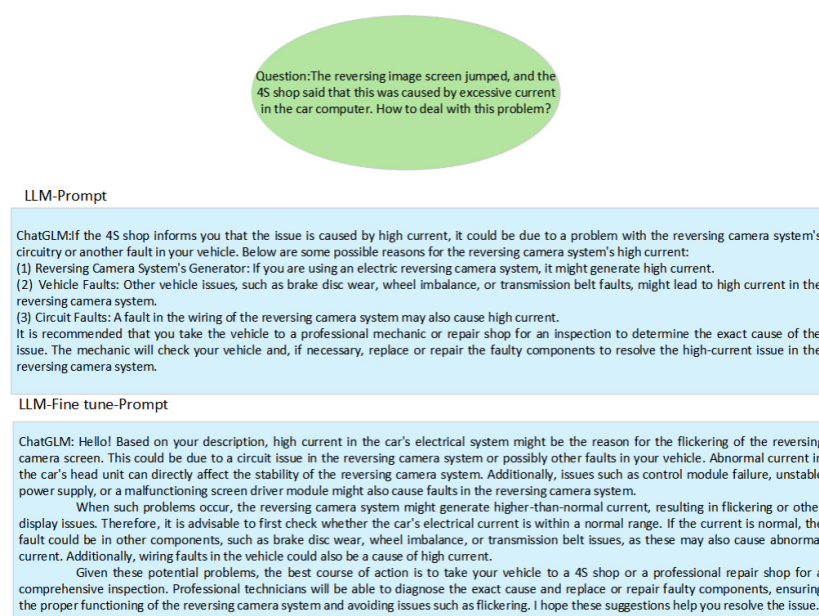


Figure 14. Comparison of fault diagnosis accuracy before and after model training.

It is evident that the trained model effectively circumvents the fallacies and illusions in the original model, manifesting a superior comprehension of NEVs fault knowledge, coupled with enhanced logical reasoning and analysis capabilities.

5.2. Discussion on Fault Data Retrieval and System Interaction

5.2.1. The Process of Intelligent Retrieval of Fault Information of NEVs

In the typical pursuit of fault data retrieval, users are generally required to initiate the process by inputting a preliminary description of the fault-related information. Subsequently, the system engages in the identification and categorization of the input, ascertaining whether the information is fault-related or not. In the event that the input is identified as non-fault-related information, it is managed through the intervention of conventional, untrained fault knowledge-based LLMs, which provide pertinent services. Should the system deduce that the user's submission pertains to a fault issue associated with NEVs, the information is then routed to the intelligent fault data search service system. The fault classification model proceeds to classify the information, ascertaining the appropriate category from among the 12 predefined fault types. Upon establishing the general category of the fault information, the fault knowledge model extracts the specific type of vehicle fault from the fault knowledge graph via an information search. Following the correlation of the specific fault phenomenon, the fault knowledge model is tasked with delivering to the user a comprehensive search outcome detailing the fault cause and corresponding resolution, thus facilitating a logical, reliable, and coherent fault reasoning analysis. If deemed necessary, the system is also capable of transmitting the entire entity, attribute, and relationship information contained within the fault knowledge graph to the user interface, thereby elucidating the complete intelligent fault data search process. Figure 15 illustrates the intelligent search query process for fault information when the state of charge of an electric vehicle's battery is low, which is to say, the ratio of the remaining battery capacity to the total battery capacity falls below an acceptable threshold.

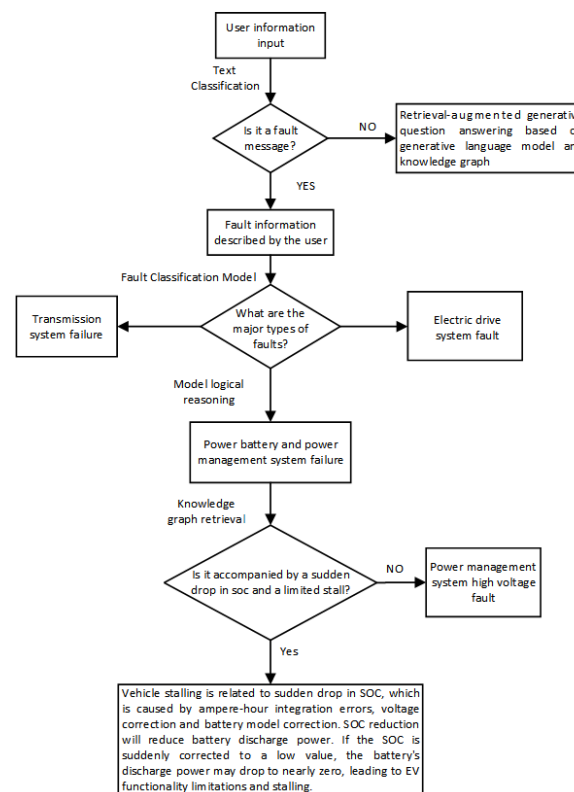


Figure 15. User fault data retrieval process.

5.2.2. Elaboration of the System Information Interaction Flow's Framework

In the functioning of the NEVs Fault Data Retrieval System, the primary dependence is placed upon a meticulously constructed knowledge base, derived from the automobile fault knowledge graph. This knowledge base facilitates the retrieval of pertinent fault data and their associated attribute relationships. Following sophisticated processing and refinement, the information is presented in the form of a Prompt to an LLM. Subsequently, leveraging the enhanced knowledge generation and reasoning capabilities of the LLM, which have been developed through rigorous training and fine-tuning, the system obtains the desired retrieval outcomes and formulates a proposed solution for the rectification of faults, as depicted in Figure 16.

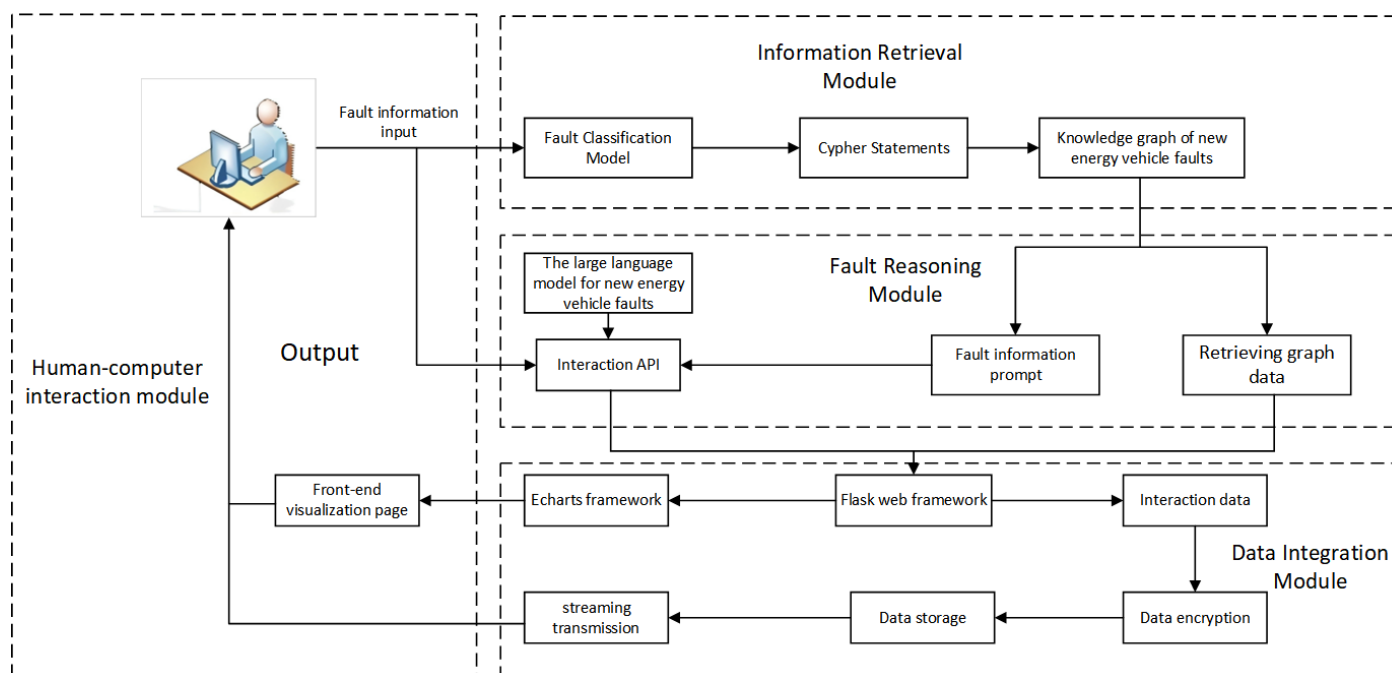


Figure 16. Fault information system interaction process.

In the initial phase, the system engages in the retrieval of fault-related information. This involves encoding the fault data of the user's NEVs and incorporating techniques such as truncation and padding to ensure compatibility with the input requirements of the model. Subsequently, the model's weights are loaded, and the encoded data is fed into the LLM for inferential processing, thereby yielding the desired output results. These output statements are then utilized to construct Cypher queries that align with the structure of the fault knowledge graph, enabling the retrieval of pertinent fault information from the Neo4j database.

During the second phase, fault data inference is conducted. To facilitate interactive services for users and to allow for the seamless integration of knowledge graph data into the interface, the system employs an API styled after OpenAI's format. This API extracts the relevant prompt from the knowledge graph and presents it to the model in a "role": "system" format. LLM then performs reasoning analysis to ascertain the cause of the fault in NEVs and to determine the appropriate countermeasures.

The third phase entails data integration. The system has the capability to encrypt the user's fault-related question-and-answer data for secure storage, and to feed back data that has received high user satisfaction with NEVs fault LLM for subsequent learning and enhancement of the model.

Finally, in the fourth phase, the system facilitates the display of fault information to the user by transmitting the data acquired from the API. Concurrently, the data sourced from the knowledge graph is visualized using the ECharts framework. This approach enables the system not only to visualize the knowledge graph but also to achieve a graphical representation of the complete reasoning chain.

6. Conclusions and Prospects

In conclusion, the development of an intelligent retrieval system for NEV fault diagnosis has successfully met the research objectives. The integration of ChatGLM3-6B with a knowledge graph and an improved fault retrieval algorithm has enhanced the system's capability. The model was fine-tuned using the LoRA methodology, leading to superior performance compared to baseline models. Additionally, a fault tree analysis method was combined with an expert system to efficiently process fault data. Comparative evaluations

demonstrated the system's effectiveness in improving retrieval accuracy and diagnostic precision.

The development of this system not only enhances the efficiency and accuracy of NEV fault diagnosis but also lays the foundation for the practical application of intelligent fault retrieval technologies. By integrating advanced generative models and knowledge graph techniques, it drives innovation in fault data processing and diagnostic algorithms, offering new technological pathways and solutions for the field.

While the system has achieved its initial goals, there is still room for improvement. Future enhancements will focus on three key areas:

- (1) Enhancing fault data retrieval—Improving entity extraction, fault categorization, and relational identification, and optimizing graph-based retrieval processes.
- (2) Automating knowledge graph updates—Leveraging generative models to enable real-time knowledge graph construction and updates, ensuring data relevance and accuracy.
- (3) System integration—Incorporating the NEV fault retrieval system with vehicular networking and maintenance decision support systems to create a comprehensive NEV service ecosystem, facilitating data interchange and functional integration.

Author Contributions: Conceptualization, H.Z.; methodology, H.Z.; software, H.Z.; validation, B.S.; formal analysis, Y.Z. and B.S.; investigation, Y.Z.; resources, H.Z.; data curation, B.S.; writing—original draft preparation, H.Z.; writing—review and editing, Y.Z. and Y.W.; visualization, H.Z. and B.S.; supervision, Z.F.; project administration, X.X.; funding acquisition, Z.F. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding

Institutional Review Board Statement: Not applicable

Informed Consent Statement: Not applicable

Data Availability Statement: The data presented in this study are openly available at <https://figshare.com/s/700fd7ffaa4df07bfc98>. Accessed on 5 April 2025.

Conflicts of Interest: The authors declare no conflict of interest.

Appendix A

The standards referenced in this study are primarily national standards related to NEVs; the following are some of the key standards referenced (Table A1).

Table A1. Partial List of National Standards for NEVs

No.	Standard (GB/T)	Title
1	GB/T 18385-2005	Electric Vehicle Power Performance
2	GB/T 18386.1-2021	Energy Consumption and Range for Light-duty Electric Vehicles
3	GB/T 18388-2005	Electric Vehicle Type Testing Regulations
4	GB/T 24552-2009	Electric Vehicle Windshield Defogging and Dew Removal
5	GB/T 28382-2012	Technical Conditions for Pure Electric Passenger Vehicles
6	GB/T 36980-2018	Energy Consumption Rate Limit for Electric Vehicles
7	GB/T 19750-2005	Hybrid Electric Vehicle Type Testing Regulations
8	GB/T 19752-2005	Hybrid Electric Vehicle Power Performance
9	GB/T 19753-2021	Energy Consumption Testing Methods for Hybrid Electric Vehicles

Please note that these are just a few of the relevant standards, and the full list of applicable national standards could not be included here due to the large number.

References

1. Khaneghah, M.Z.; Alzayed, M.; Chaoui, H. Fault Detection and Diagnosis of the Electric Motor Drive and Battery System of Electric Vehicles. *Machines* **2023**, *11*, 713.
2. Lang, W.; Hu, Y.; Gong, C.; Zhang, X.; Xu, H.; Deng, J. Artificial Intelligence-Based Technique for Fault Detection and Diagnosis of EV Motors: A Review. *IEEE Trans. Transp. Electrification* **2021**, *8*, 384–406.
3. El-Ghaname, J. Automotive Fault Detection Using Knowledge Graph Embedding. Master's Thesis, University of Windsor, Windsor, ON, Canada, 2022.
4. Cong, X.; Zhang, C.; Jiang, J.; Zhang, W.; Jiang, Y.; Zhang, L. A Comprehensive Signal-Based Fault Diagnosis Method for Lithium-Ion Batteries in Electric Vehicles. *Energies* **2021**, *14*, 1221. <https://doi.org/10.3390/en14051221>.
5. Nandi, S.; Toliyat, H.A.; Li, X. Condition Monitoring and Fault Diagnosis of Electrical Motors—A Review. *IEEE Trans. Energy Convers.* **2005**, *20*, 719–729.
6. Ahmad, I.S.; Abubakar, S.; Gambo, F.L.; Gadanya, M.S. A Rule-Based Expert System for Automobile Fault Diagnosis. *Int. J. Perceptive Cogn. Comput.* **2021**, *7*, 20–25.
7. Venkatasubramanian, V.; Rengaswamy, R.; Kavuri, S.N.; Yin, K. A Review of Process Fault Detection and Diagnosis: Part III: Process History Based Methods. *Comput. Chem. Eng.* **2003**, *27*, 327–346.
8. Zhang, Z.; Deng, Y.; Liu, X.; Liao, J. Research on Fault Diagnosis of Rotating Parts Based on Transformer Deep Learning Model. *Appl. Sci.* **2024**, *14*, 10095. <https://doi.org/10.3390/app142210095>.
9. Qiao, Z.; Lei, Y.; Li, N. Applications of Stochastic Resonance to Machinery Fault Detection: A Review and Tutorial. *Mech. Syst. Signal Process.* **2019**, *122*, 502–536. <https://doi.org/10.1016/j.ymssp.2018.12.032>.
10. Xiao, Y.; Han, F.; Ding, Y.; Liu, W. Research on Fault Diagnosis Method of Rapier Loom Based on the Fusion of Expert System and Fault Tree. *J. Intell. Fuzzy Syst.* **2021**, *41*, 3429–3441.
11. Zheng, H.; Wang, R.; Yang, Y.; Yin, J.; Li, Y.; Li, Y.; Xu, M. Cross-Domain Fault Diagnosis Using Knowledge Transfer Strategy: A Review. *IEEE Access* **2019**, *7*, 129260–129290. <https://doi.org/10.1109/ACCESS.2019.2939876>.
12. Jafari, S.; Shahbazi, Z.; Byun, Y.-C.; Lee, S.-J. Lithium-Ion Battery Estimation in Online Framework Using Extreme Gradient Boosting Machine Learning Approach. *Mathematics* **2022**, *10*, 888. <https://doi.org/10.3390/math10060888>.
13. Xia, Z.; Ye, F.; Dai, M.; Zhang, Z. Real-Time Fault Detection and Process Control Based on Multi-Channel Sensor Data Fusion. *Int. J. Adv. Manuf. Technol.* **2021**, *115*, 795–806. <https://doi.org/10.1007/s00170-020-06168-y>.
14. Du, C.; Li, W.; Rong, Y.; Li, F.; Yu, F.; Zeng, X. Research on the Application of Artificial Intelligence Method in Automobile Engine Fault Diagnosis. *Eng. Res. Express* **2021**, *3*, 026002. <https://doi.org/10.1088/2631-8695/ac01ad>.
15. Liu, H.; Song, X.; Zhang, F. Fault Diagnosis of New Energy Vehicles Based on Improved Machine Learning. *Soft Comput.* **2021**, *25*, 12091–12106. <https://doi.org/10.1007/s00500-021-05860-9>.
16. Gong, C.-S.A.; Su, C.-H.S.; Chen, Y.-H.; Guu, D.-Y. How to Implement Automotive Fault Diagnosis Using Artificial Intelligence Scheme. *Micromachines* **2022**, *13*, 1380. <https://doi.org/10.3390/mi13091380>.
17. Rogers, A.; Kovaleva, O.; Rumshisky, A. A Primer in BERTology: What We Know about How BERT Works. *Trans. Assoc. Comput. Linguist.* **2021**, *8*, 842–866.
18. Devlin, J.; Chang, M.-W.; Lee, K.; Toutanova, K. Bert: Pre-Training of Deep Bidirectional Transformers for Language Understanding. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), Minneapolis, MN, USA, 2–7 June 2019; pp. 4171–4186.
19. Dalal, A.-A.; Cai, Z.; Al-qaness, M.A.; Alawamy, E.A.; Alalimi, A. ETR: Enhancing Transformation Reduction for Reducing Dimensionality and Classification Complexity in Hyperspectral Images. *Expert Syst. Appl.* **2023**, *213*, 118971.
20. McCann, B.; Keskar, N.S.; Xiong, C.; Socher, R. The Natural Language Decathlon: Multitask Learning as Question Answering. *arXiv* **2018**, arXiv:180608730.
21. Wang, J.; Wang, C.; Luo, F.; Tan, C.; Qiu, M.; Yang, F.; Shi, Q.; Huang, S.; Gao, M. Towards Unified Prompt Tuning for Few-Shot Text Classification. *arXiv* **2022**, arXiv:220505313.
22. Radford, A.; Narasimhan, K.; Salimans, T.; Sutskever, I. Improving Language Understanding by Generative Pre-Training. **2018**. Available online: <https://openai.com/research/language-unsupervised> (accessed on 5 April 2025)
23. Nori, H.; King, N.; McKinney, S.M.; Carignan, D.; Horvitz, E. Capabilities of Gpt-4 on Medical Challenge Problems. *arXiv* **2023**, arXiv:230313375.

24. Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F.L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S. Gpt-4 Technical Report. *arXiv* **2023**, arXiv:230308774.
25. Ji, S.; Pan, S.; Cambria, E.; Marttinen, P.; Yu, P.S. A Survey on Knowledge Graphs: Representation, Acquisition, and Applications. *IEEE Trans. Neural Netw. Learn. Syst.* **2021**, *33*, 494–514.
26. Bi, Z.; Chen, J.; Jiang, Y.; Xiong, F.; Guo, W.; Chen, H.; Zhang, N. Codekgc: Code Language Model for Generative Knowledge Graph Construction. *ACM Trans. Asian Low-Resour. Lang. Inf. Process.* **2024**, *23*, 45.
27. Liang, W.; Yuksekgonul, M.; Mao, Y.; Wu, E.; Zou, J. GPT Detectors Are Biased against Non-Native English Writers. *Patterns* **2023**, *4*, 100779.
28. Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Xia, F.; Chi, E.; Le, Q.V.; Zhou, D. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Adv. Neural Inf. Process. Syst.* **2022**, *35*, 24824–24837.
29. Li, X.; Zhao, R.; Chia, Y.K.; Ding, B.; Joty, S.; Poria, S.; Bing, L. Chain-of-Knowledge: Grounding Large Language Models via Dynamic Knowledge Adapting over Heterogeneous Sources. *arXiv* **2024**, arXiv:2305.13269.
30. Sun, J.; Xu, C.; Tang, L.; Wang, S.; Lin, C.; Gong, Y.; Ni, L.M.; Shum, H.-Y.; Guo, J. Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph. *arXiv* **2023**, arXiv:230707697.
31. Gao, A. Prompt Engineering for Large Language Models. 2023. Available online: <https://ssrn.com/abstract=4504303> (accessed on 5 April 2025).
32. Liu, P.; Yuan, W.; Fu, J.; Jiang, Z.; Hayashi, H.; Neubig, G. Pre-Train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing. *ACM Comput. Surv.* **2023**, *55*, 195. <https://doi.org/10.1145/3560815>.
33. Wang, J.; Yin, W.; Gao, J. Cases Integration System for Fault Diagnosis of CNC Machine Tools Based on Knowledge Graph. *Acad. J. Sci. Technol.* **2023**, *5*, 273–281. <https://doi.org/10.54097/ajst.v5i1.5664>.
34. Wei, J.; Tay, Y.; Bommasani, R.; Raffel, C.; Zoph, B.; Borgeaud, S.; Yogatama, D.; Bosma, M.; Zhou, D.; Metzler, D. Emergent Abilities of Large Language Models. *arXiv* **2022**, arXiv:220607682.
35. Hamad, K.; Kaya, M. A Detailed Analysis of Optical Character Recognition Technology. *Int. J. Appl. Math. Electron. Comput.* **2016**, 244–249.
36. Papineni, K.; Roukos, S.; Ward, T.; Zhu, W.-J. Bleu: A Method for Automatic Evaluation of Machine Translation. In Proceedings of the Proceedings of the 40th annual meeting of the Association for Computational Linguistics, Philadelphia, PA, USA, 6–12 July 2002; pp. 311–318.
37. Lin, C.-Y. Rouge: A Package for Automatic Evaluation of Summaries. In Proceedings of the Text Summarization Branches Out, Barcelona, Spain, 25–26 July 2004; pp. 74–81.
38. Chang, Y.; Wang, X.; Wang, J.; Wu, Y.; Yang, L.; Zhu, K.; Chen, H.; Yi, X.; Wang, C.; Wang, Y.; et al. A Survey on Evaluation of Large Language Models. *ACM Trans. Intell. Syst. Technol.* **2024**, *15*, 39. <https://doi.org/10.1145/3641289>.
39. Hao, X.; Ji, Z.; Li, X.; Yin, L.; Liu, L.; Sun, M.; Liu, Q.; Yang, R. Construction and Application of a Knowledge Graph. *Remote Sens.* **2021**, *13*, 2511. <https://doi.org/10.3390/rs13132511>.
40. Hogan, A.; Blomqvist, E.; Cochez, M.; D’amato, C.; Melo, G.D.; Gutierrez, C.; Kirrane, S.; Gayo, J.E.L.; Navigli, R.; Neumaier, S.; et al. Knowledge Graphs. *ACM Comput. Surv.* **2022**, *54*, 71. <https://doi.org/10.1145/3447772>.
41. Jiang, X.-J.; Zhou, W.; Hou, J. Construction of Fault Diagnosis System for Control Rod Drive Mechanism Based on Knowledge Graph and Bayesian Inference. *Nucl. Sci. Tech.* **2023**, *34*, 21. <https://doi.org/10.1007/s41365-023-01173-8>.
42. Deng, J.; Wang, T.; Wang, Z.; Zhou, J.; Cheng, L. Research on Event Logic Knowledge Graph Construction Method of Robot Transmission System Fault Diagnosis. *IEEE Access* **2022**, *10*, 17656–17673. <https://doi.org/10.1109/ACCESS.2022.3150409>.
43. Wang, H.; Qin, K.; Zakari, R.Y.; Lu, G.; Yin, J. Deep Neural Network-Based Relation Extraction: An Overview. *Neural Comput. Appl.* **2022**, *34*, 4781–4801.
44. Ding, N.; Qin, Y.; Yang, G.; Wei, F.; Yang, Z.; Su, Y.; Hu, S.; Chen, Y.; Chan, C.-M.; Chen, W.; et al. Parameter-Efficient Fine-Tuning of Large-Scale Pre-Trained Language Models. *Nat. Mach. Intell.* **2023**, *5*, 220–235. <https://doi.org/10.1038/s42256-023-00626-4>.
45. Min, B.; Ross, H.; Sulem, E.; Veyseh, A.P.B.; Nguyen, T.H.; Sainz, O.; Agirre, E.; Heintz, I.; Roth, D. Recent Advances in Natural Language Processing via Large Pre-Trained Language Models: A Survey. *ACM Comput. Surv.* **2023**, *56*, 30. <https://doi.org/10.1145/3605943>.
46. Chai, C.P. Comparison of Text Preprocessing Methods. *Nat. Lang. Eng.* **2023**, *29*, 509–553. <https://doi.org/10.1017/S1351324922000213>.
47. Liu, X.; Ji, K.; Fu, Y.; Tam, W.L.; Du, Z.; Yang, Z.; Tang, J. P-Tuning v2: Prompt Tuning Can Be Comparable to Fine-Tuning Universally across Scales and Tasks. *arXiv* **2021**, arXiv:211007602.

48. Hu, E.J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; Chen, W. Lora: Low-Rank Adaptation of Large Language Models. *ICLR* **2022**, *1*, 3.
49. Chen, H.; Shi, N.; Chen, L.; Lee, R. Enhancing Educational Q&A Systems Using a Chaotic Fuzzy Logic-Augmented Large Language Model. *Front. Artif. Intell.* **2024**, *7*, 1404940.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.